A COMPANION VOLUME FOR ABSOLUTE BEGINNERS

Authentication, Login and Registration

How this LMS works out who you are, proves it on every request, and decides what you are allowed to do.

Django 5.2.7 | Django REST Framework 3.15.2 | Simple JWT 5.4.0 | React 19 | React Router 7 | PBKDF2 password hashing | Bearer tokens

Who this is for. Someone who has heard the words "login", "token" and "hash" but has never had them explained. No prior knowledge of Django, React or web security is assumed. Every term is spelled out the first time it appears.

What it covers. One subject only: accounts. Making them, signing in with them, staying signed in, changing a password, forgetting a password, and the rules that decide which of the three roles may do what. The rest of the app — courses, lessons, grading — lives in the main book.

How to use it. Read chapters 1 to 9 in order; they build on each other. Chapter 10 is hands-on and you should type it. Chapters 11 and 12 are references you come back to when something breaks or when you are getting ready to put the app on a real server.

Contents

CHAPTER 1

What Authentication Actually Is

Before any code, the idea. If you understand this chapter, the rest of the book is detail.

The two questions

Every app that has accounts asks two questions, in this order, and it is worth being strict about which is which because beginners mix them up constantly.

QUESTION	NAME	IN THIS APP
Who are you, and can you prove it?	**Authentication** (often shortened to *authn*)	You type a phone number and a password. The server checks them and hands you a token.
Now that I know who you are, what are you allowed to do?	**Authorisation** (*authz*)	Your account has a role — admin, teacher or student — and each role may change different things.

Authentication comes first and only happens properly once, at login. Authorisation happens on *every single request* afterwards. A signed-in student and a signed-in admin are equally authenticated; they are not equally authorised.

> NOTE
>
> The two words are so close in spelling that the shorthand `authn` and `authz` exists purely so people can tell them apart out loud. Chapters 2 to 7 of this book are about authn. Chapter 8 is about authz.

The doorman and the floor plan

Picture an office building. At the front desk a doorman checks your face against your ID card. That is authentication: he is satisfied you are who you say you are. He gives you a plastic badge on a lanyard.

The badge does not open every door. It opens the doors your job needs. The server room reader beeps red at a visitor badge and green at an engineer's. That is authorisation, and notice that it is checked at each door, not at the front desk. Nobody re-examines your face at the server room; the badge speaks for you.

The token this app gives you at login is that badge. The permission classes on each API endpoint are the card readers on each door. Hold on to this picture — it is not a simplification for beginners, it is genuinely how the code is arranged.

The web forgets you instantly

Here is the fact that makes all of this necessary, and almost nobody explains it first.

HTTP is stateless. HTTP is the language browsers and servers speak. *Stateless* means the server keeps no memory between one request and the next. Each request arrives as a complete stranger. If you log in, and then a second later ask for the list of courses, the server has no idea those two things came from the same person. As far as it is concerned, two unrelated strangers knocked on the door.

That sounds broken until you see the upside: a stateless server can be switched off, restarted, or duplicated across ten machines and nothing personal is lost, because nothing personal was being held.

So every request that needs to know who you are has to **carry the proof with it**. Not "log in and then be remembered" — log in, get proof, and re-present that proof every single time. Twenty API calls means twenty presentations of the badge.

> COMMON TRAP
>
> "I logged in, so why does the server say 401?" Almost always: the request went out without the proof attached. Being logged in is not a state the server holds. It is a piece of data your browser has to send. Chapter 7 shows exactly where that attaching happens (`api.js`), and Chapter 11 lists the ways it fails.

Two ways to be remembered

There are two mainstream answers to "what proof do I carry?", and it helps to know both, because tutorials will show you the other one and you should not be confused by it.

Sessions and cookies

The server writes a row in a table: *session* `a3f9…` *belongs to user 7*. It sends the browser a small text file called a **cookie** holding only `a3f9…`. The browser sends that cookie back automatically on every request. The server looks it up and finds user 7.

The proof is a claim ticket. It means nothing on its own; the meaning is in the server's table. Django's own admin site at `/admin/` works exactly this way, which is why you can be signed into `/admin/` in one tab and signed out of the React app in another. They use different mechanisms.

Tokens

The server writes nothing down. It hands you a note that already says who you are, and signs it so it cannot be altered. On each request you present the note. The server checks the signature, believes the contents, and moves on.

The proof is a signed statement, not a claim ticket. That is a **JSON Web Token**, or **JWT**, and it is what this project uses.

	SESSION + COOKIE	TOKEN (JWT)
Server stores anything?	Yes, one row per session	No
Sent by the browser	Automatically	Only if your code attaches it
Logging one person out	Delete the row. Instant.	Hard. See Chapter 6.
Works well for	A server rendering its own HTML pages	A separate frontend, or a phone app, talking to an API
Used here for	Django's `/admin/` site	Everything the React app does

What this project chose

This app is in two halves that run as two separate programs: a Django backend on `http://127.0.0.1:8001` and a React frontend on `http://localhost:5180`. The frontend is not pages produced by Django; it is its own app that phones the backend for data. That split is what makes tokens the natural fit, and it comes with three consequences you will meet later:

- The frontend must attach the token by hand to every call. Nothing is automatic.
- Two different addresses means the browser's **same-origin** safety rule applies, which is why `corsheaders` is installed. CORS is the server saying "requests from that other address are fine by me".
- There is no real "log out everywhere" button, because there is no session table to empty.

The whole story in nine lines

Here is the entire subject of this book, start to finish. Every later chapter is one of these lines, slowed down.

```
1. An admin fills in the Accounts form.        -> POST /api/register/
2. Django hashes the password, saves a User
   row and a Profile row holding the phone
   number and the role.
3. The new person opens /login and types
   their phone number and password.            -> POST /api/login/
4. Django finds the Profile by phone, checks
   the password against the stored hash.
5. Django signs two tokens and sends them
   back, along with the person's role.
6. The browser stores the access token in
   localStorage.
7. Every later call attaches it as a header:
   Authorization: Bearer <token>
8. Django verifies the signature, sets
   request.user, and the endpoint's permission
   class checks the role.
9. Eight hours later the token expires and
   step 3 happens again.
```

The files involved

Nine files do all of it. You will meet each in turn; this table is here so you always know where you are.

FILE	ITS JOB IN ONE LINE	CHAPTER
`backend/backend/models.py`	The `Profile` table: phone number and role	2
`backend/backend/serializers.py`	Checks and shapes what comes in and goes out	3, 9
`backend/backend/views.py`	The register, login, profile and password endpoints	3, 5, 9
`backend/backend/urls.py`	Which web address reaches which view	3
`backend/backend/permissions.py`	Which role may change what	8
`backend/lms/settings.py`	Token lifetimes, password rules, the default lock	6, 8
`frontend/src/api.js`	Stores the token, attaches it to every call	7
`frontend/src/AuthContext.jsx`	Holds "am I logged in" for the whole React app	7
`frontend/src/components/ProtectedRoute.jsx`	Bounces signed-out visitors to `/login`	7

What you have now

You can say what authentication is and how it differs from authorisation. You know why a stateless web forces you to carry proof on every request, what the two common kinds of proof are, and which one this project uses. You have seen the nine-step flow you are about to study one step at a time, and you know which file each step lives in.

Next: the two database tables that a person is made of.

CHAPTER 2

Where a Person Is Stored

An account in this app is not one row in one table. It is two rows in two tables, joined together. Knowing which half holds what saves you hours later, because almost every confusing bug in this area comes from looking in the wrong half.

Django brings a User table with it

You did not write the `User` model. Django ships with it, in `django.contrib.auth`, and it is switched on by these two lines that were already in `INSTALLED_APPS` when the project was created:

backend/lms/settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',          # <- the User model and password hashing
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'backend',
    'rest_framework',
    'rest_framework_simplejwt.token_blacklist',
    'corsheaders',
]
```

Because it is already there, the code refers to it rather than defining it:

backend/backend/serializers.py

```
from django.contrib.auth import get_user_model

User = get_user_model()
```

`get_user_model()` asks Django "whichever model is currently in charge of accounts, give me that one". Today it returns the built-in `User`. If the project ever swapped in a custom user model, this line would keep working while `from django.contrib.auth.models import User` would quietly point at the wrong table. It is the habit worth copying.

What is in the User table

The columns you will actually touch:

COLUMN	HOLDS	NOTE
`id`	1, 2, 3 …	Django adds it. This number ends up inside your token.
`username`	`nadia`	Unique. In this app it is a *label*, not what you log in with.
`password`	`pbkdf2_sha256$1000000$…`	Never the password. A hash of it. Chapter 4.
`email`	`nadia@example.com`	Used by the forgotten-password flow.
`first_name`, `last_name`	Display name	Editable by the person themselves.
`is_active`	`True`	The password reset flow refuses inactive accounts.
`is_superuser`	`False`	Set by `createsuperuser`. Treated as admin here.

What is *not* in that table: a phone number, and a role. Django has no opinion about either. That is the reason for the second table.

Why a second table: Profile

You need two facts Django's `User` does not carry, so you store them next to it:

backend/backend/models.py

```python
from django.db import models
from django.contrib.auth.models import User

class Profile(models.Model):
    ADMIN = "admin"
    TEACHER = "teacher"
    STUDENT = "student"

    ROLE_CHOICES = [
        (ADMIN, "Admin"),
        (TEACHER, "Teacher"),
        (STUDENT, "Student"),
    ]

    user = models.OneToOneField(User, on_delete=models.CASCADE)
    phone = models.CharField(max_length=20, unique=True)
    role = models.CharField(max_length=10, choices=ROLE_CHOICES, default=STUDENT)

    def __str__(self):
        return f"{self.user.username} – {self.phone} ({self.role})"
```

An account is therefore a pair:

```
auth_user                          backend_profile
---------                          ---------------
id        7                        id       4
username  nadia            <-----  user_id  7
password  pbkdf2_sha256$...        phone    01711110001
email     nadia@...                role     teacher
is_active True
```

The phone number is the interesting one. **You log in with the phone number, not the username and not the email.** That surprises everybody the first time, and it is worth saying out loud now because Chapter 5 depends on it: the login view searches `Profile` for the phone, then follows the link to the `User` to check the password.

OneToOne, and what it means

`models.OneToOneField(User, ...)` says: one Profile belongs to exactly one User, and one User has at most one Profile. Not two, not five. The database enforces it — try to give a second Profile to the same User and it refuses.

The relationship works in both directions:

```
profile.user          # the User, from the Profile
user.profile          # the Profile, from the User  (raises if there isn't one)
```

`on_delete=models.CASCADE` answers "what if the User is deleted?" — cascade means the Profile goes with it. That is the right answer here: a phone number and a role belonging to a deleted account are meaningless leftovers.

> **COMMON TRAP**
>
> `user.profile` throws `RelatedObjectDoesNotExist` when there is no Profile row, which reads like a crash rather than a missing value. That is why the safety-conscious code in this project writes `Profile.objects.filter(user=user).first()` instead, which returns `None` and lets the caller decide. You will see both spellings; now you know why.

unique=True does real work

`phone = models.CharField(max_length=20, unique=True)` puts a rule in the database itself: no two Profile rows may hold the same phone number.

This is not tidiness. It is the thing that makes logging in by phone possible at all. The login view runs `Profile.objects.get(phone=phone)` — `get`, singular. If two accounts shared a number, that call would raise `MultipleObjectsReturned` and nobody could sign in. The uniqueness rule is load-bearing.

The same rule is checked twice more, higher up, so the user sees a sentence instead of a database error: once when an admin creates an account (Chapter 3) and once when somebody edits their own phone number (Chapter 9).

The role field

`role` is a text column that may only hold one of three strings. The pattern is worth learning because Django code uses it everywhere:

- The constants `ADMIN`, `TEACHER`, `STUDENT` exist so the rest of the codebase writes `Profile.ADMIN` instead of the string `"admin"`. A typo in a constant name is an immediate error; a typo in a string is a silent bug.
- `ROLE_CHOICES` pairs each stored value with a label for humans. Django's admin turns it into a dropdown automatically; the Accounts page in React writes its own.
- `default=STUDENT` means an account with nothing specified is the least powerful kind. That direction matters. Defaults should fail towards *less* access, never more.

> **NOTE** The comment above `role` in `models.py` still says "Anyone can register, so a new account gets the least privileged role." That was true of an earlier version. Registration is admin-only now (Chapter 3) and the admin picks the role, but the safe default is kept anyway, because it is what protects rows created by any other route.

The account with no Profile

There is one way to end up with a User and no Profile, and every beginner meets it on day one: `python manage.py createsuperuser`. That command belongs to Django, knows nothing about your `Profile` model, and so creates the `User` row alone.

The account then has no phone number, and the login screen asks for a phone number. So **your brand new superuser cannot log into the React app.** Nothing is broken; the row it needs simply is not there yet. The fix is in the next section.

The code plans for this. In `permissions.py`:

backend/backend/permissions.py

```
def role_of(user):
    if not user or not user.is_authenticated:
        return None

    if user.is_superuser:
        return Profile.ADMIN

    profile = Profile.objects.filter(user=user).only("role").first()
    if profile is None:
        return Profile.STUDENT

    return profile.role
```

Three answers, in order: nobody signed in gets `None`; a superuser counts as an admin even with no Profile; anyone else missing a Profile gets the weakest role rather than an exception. Read that last line as a policy, not a fallback — when the code is unsure who you are, it gives you less, not more.

Making a Profile by hand

Once, at the very start of a fresh install, you have to create a Profile yourself. After that the register endpoint does it for you every time.

```
python manage.py createsuperuser
python manage.py runserver 8001
```

Then open `http://127.0.0.1:8001/admin/` and sign in with the username and password you just chose. This is Django's own admin site, and it uses the cookie-and-session mechanism from Chapter 1, which is why it lets you in without a phone number.

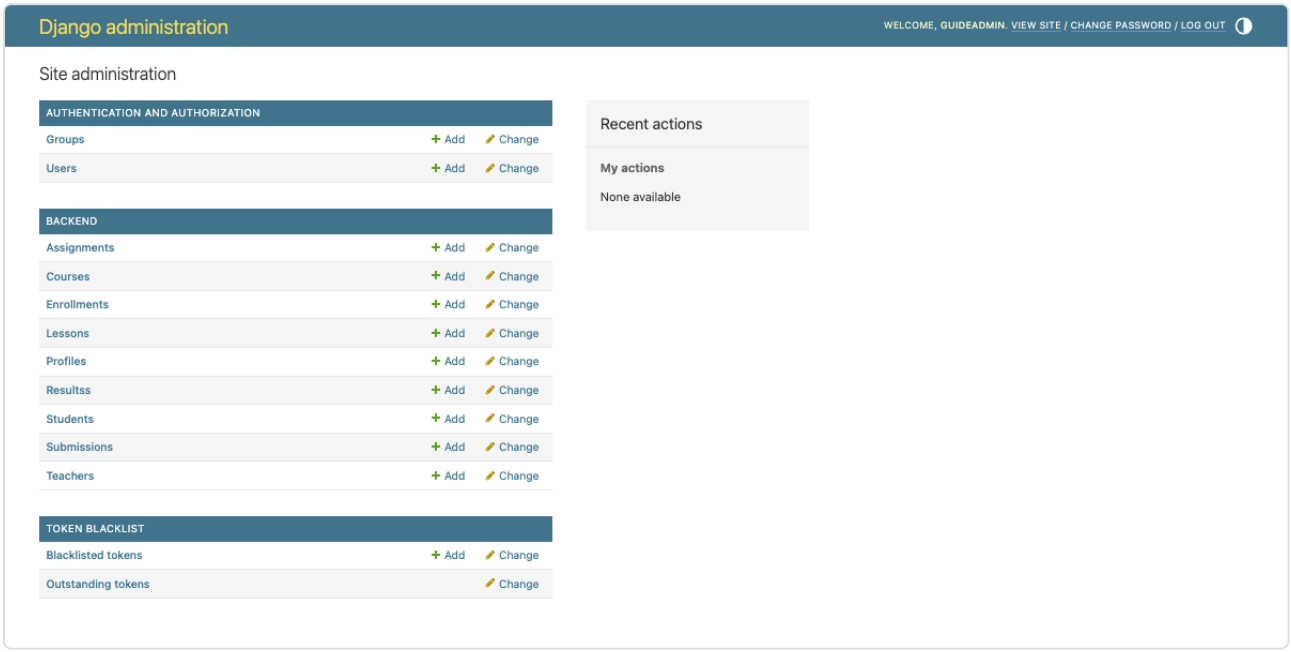

Figure 2.1 The Django admin home page. `Profiles` is in the list because `admin.py` registers it.

Click **Profiles**, then **Add profile**, and fill in three fields: your user in the dropdown, a phone number you will remember, and the role **Admin**.

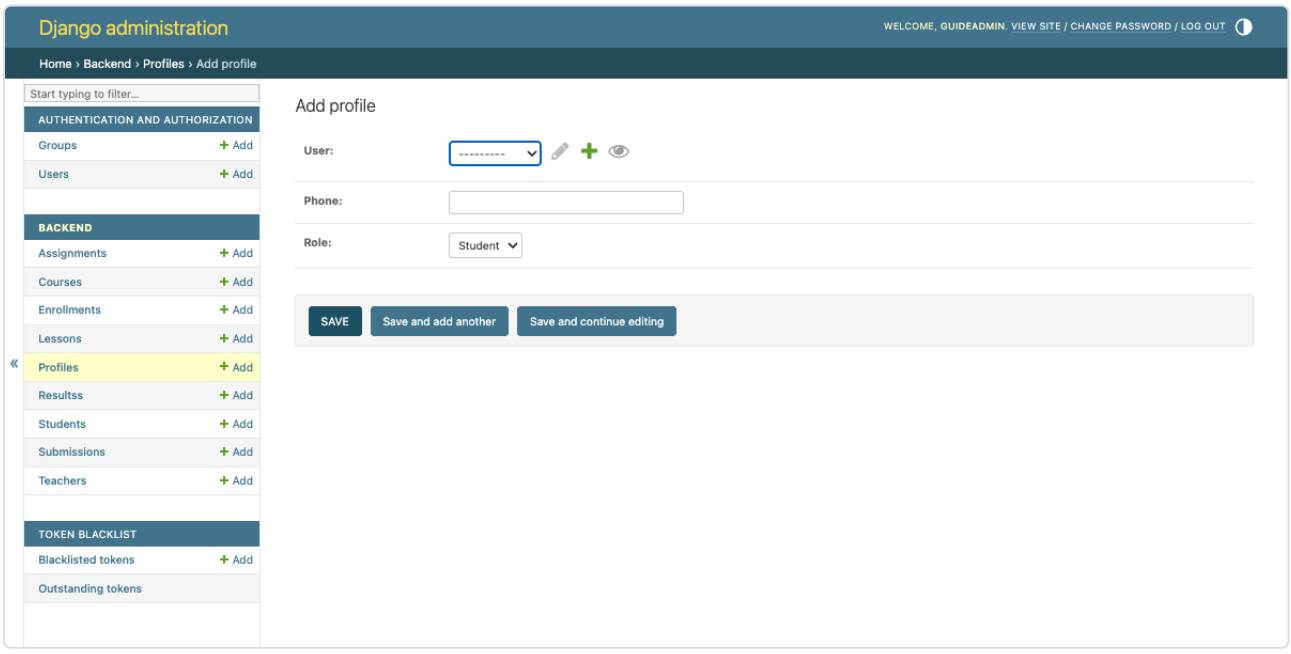

Figure 2.2 Three fields, and the account can finally log in. The phone number here is what you type on the login screen.

> **CHECKPOINT** Start the React app, go to `/login`, and sign in with the phone number you just typed into that form and your superuser password. If you land on the dashboard, both halves of an account exist and are joined correctly. If you get `Invalid phone or password`, the phone number in the Profile does not match what you typed — check it character by character in the admin.

What you have now

You know an account is two rows: Django's `User`, holding the username, the email and the hashed password, and your `Profile`, holding the phone number and the role. You know the two are joined one-to-one, that the phone number is unique because logging in depends on it, and that a superuser starts life with only half an account until you add the Profile.

Next: how that pair of rows gets created properly, by the register endpoint.

CHAPTER 3

Registration: How an Account Gets Made

Registration is the moment the two rows from Chapter 2 come into existence. In most tutorials it is a public form anybody can fill in. Here it is not, and the difference is a deliberate design decision worth understanding before the code.

There is no public sign-up

A student does not make their own account. An admin makes it for them and picks the role at the same time. There is no sign-up link on the login screen, and the old `/register` page now just redirects to `/login`:

frontend/src/App.jsx

```
{/* /register used to be a public page. There is no public sign-up any
    more: an admin creates accounts at /accounts, inside the app. */}
<Route path="/register" element={<Navigate to="/login" replace />} />
```

Why do it this way? Because of what a role is worth. In a school system, the difference between a student account and a teacher account is the difference between reading grades and setting them. If anybody could sign up, then either everyone starts as a student and an admin has to promote them one by one anyway, or the role is self-declared, which is no protection at all. Handing accounts out closes the question.

The trade-off, and it is a real one: someone has to be an admin before anybody can be anything. That is the chicken-and-egg problem Chapter 2 solved with `createsuperuser` plus a hand-made Profile.

The endpoint

Two lines connect a web address to the code that answers it.

backend/lms/urls.py

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('backend.urls')),
]
```

backend/backend/urls.py

```
urlpatterns = [
    path('login/', LoginView.as_view(), name='login'),
    path('register/', RegisterView.as_view(), name='register'),
    path('profile/', ProtectedView.as_view(), name='protected'),
    path('change-password/', ChangePasswordView.as_view(), name='change-password'),
    path("password-reset/", PasswordResetRequestView.as_view()),
    path("password-reset-confirm/", PasswordResetConfirmView.as_view()),
    # ... the eight resource endpoints follow
]
```

Read them together: `api/` from the first file plus `register/` from the second makes `/api/register/`. Six addresses in that list are the entire subject of this book.

And the view itself is almost nothing:

backend/backend/views.py

```
class RegisterView(generics.CreateAPIView):
    """Create an account. Admins only.

    There is no public sign-up: a student does not make their own account, an
    admin makes it for them and picks the role. Anonymous callers get a 401,
    signed-in teachers and students a 403.

    If there is no admin left to do this, `python manage.py createsuperuser`
    still works, and a superuser counts as an admin.
    """

    queryset = User.objects.all()
    serializer_class = RegisterSerializer
    permission_classes = [IsAdmin]
```

Three attributes and no method bodies. `generics.CreateAPIView` is a ready-made class from Django REST Framework that knows how to answer a POST request by creating something. You tell it what to create with (`serializer_class`) and who may do it (`permission_classes`), and it supplies the rest. `IsAdmin` is examined in Chapter 8; for now, read it as "admins only, no exceptions".

What a serializer is

All the real work is in the serializer, so it is worth defining the word properly.

Data lives in two shapes. In the database it is rows and columns. Over the web it is **JSON**: text made of `{` curly brackets `}`, quoted names and values, which every language can read. A **serializer** is the translator between those two shapes, and it works in both directions:

- **Coming in** — take the JSON the browser sent, check every field is present, the right type and acceptable, then create or update rows. This is where validation lives.
- **Going out** — take rows and turn them into JSON, choosing which fields the outside world is allowed to see.

That second half matters as much as the first. The password column is never in the outgoing JSON, and that is a decision written down in the serializer.

RegisterSerializer, field by field

backend/backend/serializers.py

```
class RegisterSerializer(serializers.ModelSerializer):
    phone = serializers.CharField(required=True, write_only=True)
    first_name = serializers.CharField(required=True)
    email = serializers.EmailField(required=True)
    # Only an admin can reach this endpoint, so letting the caller pick the
    # role is safe. Left out, the account is a student.
    role = serializers.ChoiceField(
        choices=Profile.ROLE_CHOICES,
        required=False,
        default=Profile.STUDENT,
    )

    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'password', 'phone', 'first_name',
'last_name', 'role']
        read_only_fields = ['id']
        extra_kwargs = {'password': {'write_only': True}}
```

`ModelSerializer` means "look at the model named in `Meta` and work out most of the fields yourself". Because `model = User`, the fields `username`, `email`, `password`, `first_name` and `last_name` come for free with the right types and lengths.

The four fields written out by hand are the ones that need something extra:

FIELD	WHY IT IS WRITTEN OUT
`phone`	It is not a `User` column at all — it belongs to `Profile`. Declaring it here lets it arrive in the same request.
`first_name`	Django's version allows blank. This app insists on one, so the field is redeclared as `required=True`.
`email`	Same reason. An email is needed later for the password reset flow, so it cannot be optional.
`role`	Also a `Profile` column. `ChoiceField` restricts it to the three known values; anything else is rejected before any row is written.

Three keywords in that block decide what is visible:

- `write_only=True` on `phone` and, via `extra_kwargs`, on `password`: these may come *in* but are never sent back *out*. This is the single most important line in the file. Without it, creating an account would answer with the password in the response.
- `read_only_fields = ['id']`: the database picks ids. A caller trying to choose one is ignored.
- `required=False, default=Profile.STUDENT` on `role`: leave it out and you get the weakest role, matching the model default from Chapter 2.

The three validate_ methods

DRF has a naming convention: define a method called `validate_<fieldname>` and it is called automatically with that field's value. Return the value to accept it, raise `ValidationError` to reject it with a message.

backend/backend/serializers.py

```
    def validate_phone(self, value):
        if Profile.objects.filter(phone=value).exists():
            raise serializers.ValidationError("This phone number is already registered.")
        return value

    def validate_username(self, value):
        if User.objects.filter(username=value).exists():
            raise serializers.ValidationError("That username is already taken.")
        return value

    def validate_password(self, value):
        # Run Django's own password rules, the same ones the reset flow uses.
        validate_password(value)
        return value
```

The first two would be caught by the database anyway, since both columns are unique. Doing it here turns a 500 crash into a 400 with a sentence a human can act on. That is the whole point: the database protects the data, the serializer protects the person using the form.

The third hands the password to Django's own rule set — the four validators listed in `settings.py`, covered in Chapter 4. Note the import at the top of the file that makes this possible:

```
from django.contrib.auth.password_validation import validate_password
```

create(), and the atomic transaction

Validation passed. Now two rows have to appear.

backend/backend/serializers.py

```
    @transaction.atomic
    def create(self, validated_data):
        phone = validated_data.pop('phone')
        email = validated_data.pop('email')
        role = validated_data.pop('role', Profile.STUDENT)
        first_name = validated_data.pop('first_name', '')
        last_name = validated_data.pop('last_name', '')

        user = User.objects.create_user(
            username=validated_data['username'],
            email=email,
            password=validated_data['password'],
            first_name=first_name,
            last_name=last_name,
        )
        Profile.objects.create(user=user, phone=phone, role=role)
        return user
```

Read the decorator first. `@transaction.atomic` wraps everything inside the method in a database transaction: either all of it is saved or none of it is. If creating the Profile fails — a duplicate phone number that slipped past validation, a lost database connection — the User created a line earlier is rolled back too.

Without it, a failure halfway through would leave a User with no Profile: an account with a password and no way to log in, and a username now taken so the admin cannot even retry with the same name. Atomic makes the pair of rows an all-or-nothing unit, which is exactly what they are.

`validated_data.pop(...)` takes a key out of the dictionary and hands it to you. The five popped fields are the ones that either belong to `Profile` or need to be passed explicitly, and popping them keeps them from being passed twice.

create_user versus create

This is the difference that matters most in the whole chapter.

	`User.objects.create(...)`	`User.objects.create_user(...)`
What it does with `password`	Stores the string exactly as given	Hashes it first, stores the hash
Result in the database	`opensesame`	`pbkdf2_sha256$1000000$…`
Can the person log in?	No — the check compares a hash to plain text	Yes
Is the password readable by anyone with database access?	Yes. This is the classic disaster.	No

COMMON TRAP

Using `User.objects.create()` for accounts is one of the most common beginner mistakes in Django, and it fails in a way that looks like a login bug rather than a security bug: the account is created, everything reports success, and then the password never works. If you ever see that symptom, look for `create` where `create_user` should be.

to_representation: putting role in the answer

backend/backend/serializers.py

```
    def to_representation(self, instance):
        data = super().to_representation(instance)
        data['role'] = instance.profile.role
        return data
```

`to_representation` is the outgoing half of the translation: rows to JSON. The default version would look for `role` on the `User`, not find it, and leave it out. This override runs the default first with `super()`, then reaches across the one-to-one link — `instance.profile.role` — and adds the value by hand.

So the admin filling in the form gets confirmation of the role they picked, in the response, without a second request.

The request and the answer

What actually travels. Sent:

```
{
  "username": "rina",
  "email": "rina@example.com",
  "password": "correct-horse-9",
  "phone": "01711110002",
  "role": "teacher",
  "first_name": "Rina",
  "last_name": "Haque"
}
```

Received, with status `201 Created`:

```
{
  "id": 8,
  "username": "rina",
  "email": "rina@example.com",
  "first_name": "Rina",
  "last_name": "Haque",
  "role": "teacher"
}
```

Look at what is missing: no `password`, no `phone`. Both were `write_only`. The password is gone because it must be; the phone is gone because the serializer treats it as an input field only.

And when something is wrong, status `400 Bad Request` with the field names as keys:

```
{
  "phone": ["This phone number is already registered."],
  "password": ["This password is too common."]
}
```

Every message is a list, because one field can fail more than one rule at a time. The frontend flattens that shape into readable lines in `readError()`, which you meet in Chapter 7.

The Accounts page

The form an admin actually uses lives at `/accounts`. It is an ordinary React form; the only line that talks to the backend is the call to `register()`.

frontend/src/pages/Accounts.jsx

```
    try {
      await register({
        username,
        email,
        password,
        phone,
        role,
        first_name: firstName,
        last_name: lastName,
      });

      // Deliberately not logged in as the new person: the admin who filled
      // this in stays signed in as themselves.
      setNotice(
        `Account created for ${username} as a ${role}. They log in with the phone number
${phone}.`,
      );
      clearForm();
    } catch (problem) {
      setError(problem.message);
    } finally {
      setIsSaving(false);
    }
```

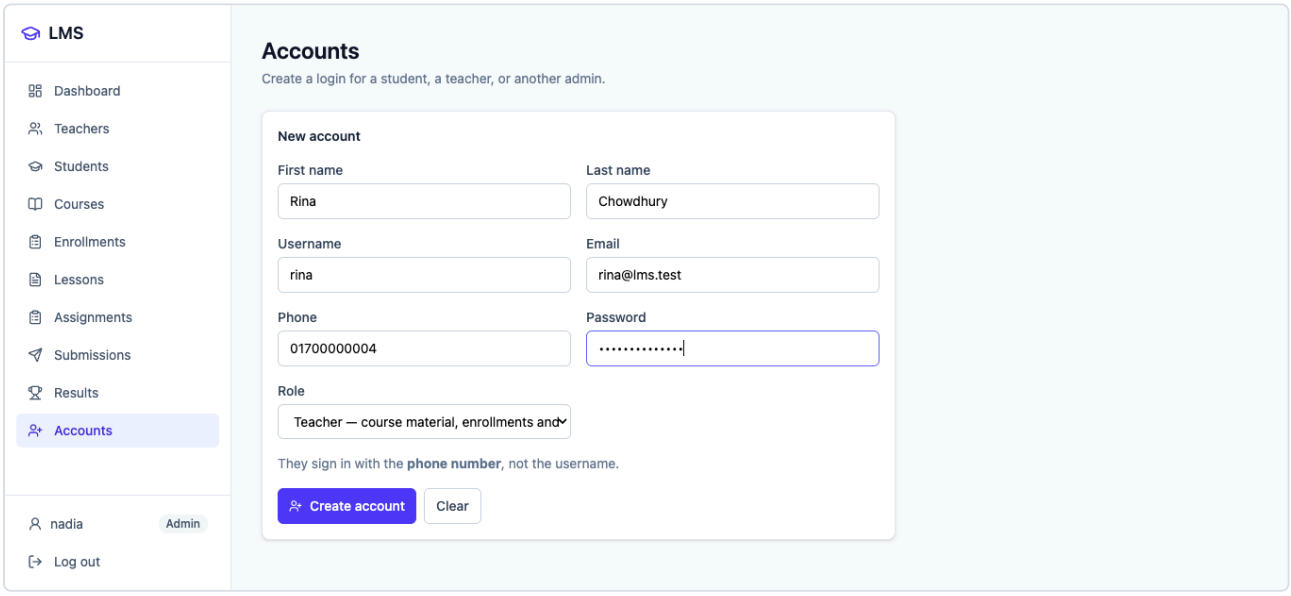

Figure 3.1 The Accounts page. The green banner repeats the phone number on purpose, because that is what the new person signs in with and it is the detail everyone forgets.

Two details in that snippet are worth copying into your own projects. First, the app does *not* log you in as the account you just created — the admin stays the admin. Second, the success message repeats the phone number, because the admin is about to have to tell somebody how to log in.

And the call itself:

frontend/src/api.js

```
// Creating an account is an admin job, so this call carries the admin's own
// token. There is no public sign-up.
export function register(details) {
  return request("/register/", {
    method: "POST",
    body: JSON.stringify(details),
  });
}
```

Compare it with `login()` in the same file, which passes `auth: false`. Registration is the opposite: it is one of the calls that *must* carry a token, because the server has to know which admin is asking.

> **CHECKPOINT** Signed in as your admin, create a teacher account. Then try to create a second account with the same phone number and confirm the red banner reads "This phone number is already registered." One form, both halves of the serializer — the happy path and the rejection.

What you have now

You know why registration is admin-only here, which two lines make `/api/register/` exist, and what a serializer is for. You have read `RegisterSerializer` field by field: which fields are declared by hand and why, what `write_only` protects, how the three `validate_` methods turn crashes into sentences, why `create()` is atomic, and the crucial difference between `create` and `create_user`.

That last point deserves its own chapter. Next: what actually happens to a password.

CHAPTER 4

Passwords: Hashing and the Vague Error

You have seen `create_user` hash a password and you have seen `check_password` verify one. This chapter opens both up. It is short, it involves no new files, and it is the part of the subject that a beginner is most likely to get catastrophically wrong in their own project.

The rule with no exceptions

A password is never stored. Not encrypted, not encoded, not hidden in a column nobody looks at. Never stored at all.

The reason is not that your database is likely to be stolen. It is that people reuse passwords. The account in your teaching LMS very probably shares a password with somebody's email, and their email is the master key to everything else they own. Storing it readably makes you the weak link in somebody else's whole life. That is why the rule has no exceptions, even for a project nobody will deploy.

So how can the server check a password it does not have? By storing something derived from it that cannot be turned back.

What a hash is

A **hash function** is a one-way calculation. Feed it any text and it returns a fixed-length scramble. Three properties make it useful:

1. **The same input always gives the same output.** That is what makes checking possible.
2. **You cannot work backwards.** Given the output there is no calculation that recovers the input. The only attack is guessing an input, hashing it, and comparing.
3. **A tiny change to the input changes the output completely.** No partial credit, no "warm" hashes.

So logging in works like this: hash what the person just typed, compare it to the stored hash. Equal means the passwords were equal. At no point did the server know either one.

For passwords specifically there is a fourth requirement, and it is the one that surprises people: **the hash must be slow on purpose**. A fast hash lets an attacker who steals the database try billions of guesses a second. A password hash is deliberately built to take a noticeable fraction of a second, which is nothing when you log in once and ruinous when you are trying a dictionary.

What Django actually stores

Here is a real row from this project's `password` column, produced by hashing the word `opensesame`:

```
pbkdf2_sha256$1000000$z5Chg0kZql1bmLAAHi3Ws7$tPM4DckZXkyDDVZmZHe4uV7lyF/509Ox6GKw+i9ZlxY=
```

It is one string, cut into four parts by `$`:

PART	VALUE HERE	MEANING
Algorithm	`pbkdf2_sha256`	Which hash function was used. Django 5.2's default.
Iterations	`1000000`	It was run a million times over. This is the deliberate slowness.
Salt	`z5Chg0kZql1bmLAAHi3Ws7`	Random text generated when the password was set.
Hash	`tPM4DckZ…lxY=`	The actual result.

Storing the settings alongside the result is what lets Django raise the iteration count in a future version without breaking anybody: each row remembers how it was made. When you next log in, Django notices the row is below the current standard, and quietly re-hashes your password at the new setting.

The salt, and why it is there

Hash the same word twice and look carefully:

```
pbkdf2_sha256$1000000$z5Chg0kZql1bmLAAHi3Ws7$tPM4DckZXkyDDVZmZHe4uV7lyF/509Ox6GKw+i9ZlxY=
pbkdf2_sha256$1000000$gP9O5ZrDz5bM00XoM0RukX$pzjvPhl9jFYCKhBCjD6Ya5doVL9W1o2xjO18WJ5LQpg=
```

Same password. Completely different stored strings, because each got a fresh random **salt** which is mixed in before hashing. Property 1 from the last section still holds — same input *and same salt* gives the same output — which is why the salt has to be stored in plain sight. It is not a secret. It has a different job.

Two jobs, in fact:

- **Identical passwords stop looking identical.** Without salt, everyone who chose `123456` would have the same row, and one glance down the column would tell an attacker which accounts to try first.
- **Precomputed tables become useless.** Attackers keep enormous lists of common passwords already hashed. Those lists are worthless against a salt they have never seen, because every password must now be attacked separately.

Checking a password

All of that is invisible from the code, which is the point of a good library. Setting:

```
user = User.objects.create_user(username="rina", password="correct-horse-9")
# or, on an account that already exists:
user.set_password("a-new-one")
user.save()          # set_password does NOT save. Forgetting this is a classic.
```

Checking, which is the single line the login view leans on:

```
user.check_password("correct-horse-9")     # True
user.check_password("Correct-horse-9")     # False - one capital letter
```

`check_password` reads the algorithm, iterations and salt out of the stored string, hashes the attempt the same way, and compares. It returns a plain `True` or `False` — never a reason, never "close".

Both spellings appear in this project. `create_user` in `RegisterSerializer` (Chapter 3), `set_password` in the change-password and reset views (Chapter 9), and `check_password` in `LoginView` and in `ChangePasswordSerializer`.

> **COMMON TRAP** `set_password()` changes the object in memory only. Without the `user.save()` on the next line, the new password is forgotten the moment the request ends and the old one still works — with no error anywhere to tell you. Look at `ChangePasswordView` in Chapter 9 and you will see the pair, always together.

The four password validators

Hashing protects a password after it is chosen. Validators are about it being worth protecting in the first place.

backend/lms/settings.py

```
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
```

VALIDATOR	REFUSES	THE MESSAGE
UserAttributeSimilarity	A password too close to the username, email, first or last name	"The password is too similar to the username."
MinimumLength	Fewer than 8 characters	"This password is too short. It must contain at least 8 characters."
CommonPassword	Anything in Django's list of 20,000 most common passwords	"This password is too common."
NumericPassword	Digits only, like `19283746`	"This password is entirely numeric."

These do not run by themselves. Something has to call `validate_password()`, and in this project three places do: `RegisterSerializer.validate_password`, `ChangePasswordSerializer.validate`, and `PasswordResetConfirmSerializer.validate`. Every route by which a password can be set runs the same rules, which is exactly the property you want — a rule enforced in two of three places is a rule that does not exist.

> **NOTE** Two of the calls pass the user as well: `validate_password(new_password, user=...)`. That extra argument is what lets UserAttributeSimilarity do its job — without a user to compare against, it has nothing to say. In the register serializer the account does not exist yet, so the check runs without it.

Why the error is deliberately vague

Get your login wrong and the app says:

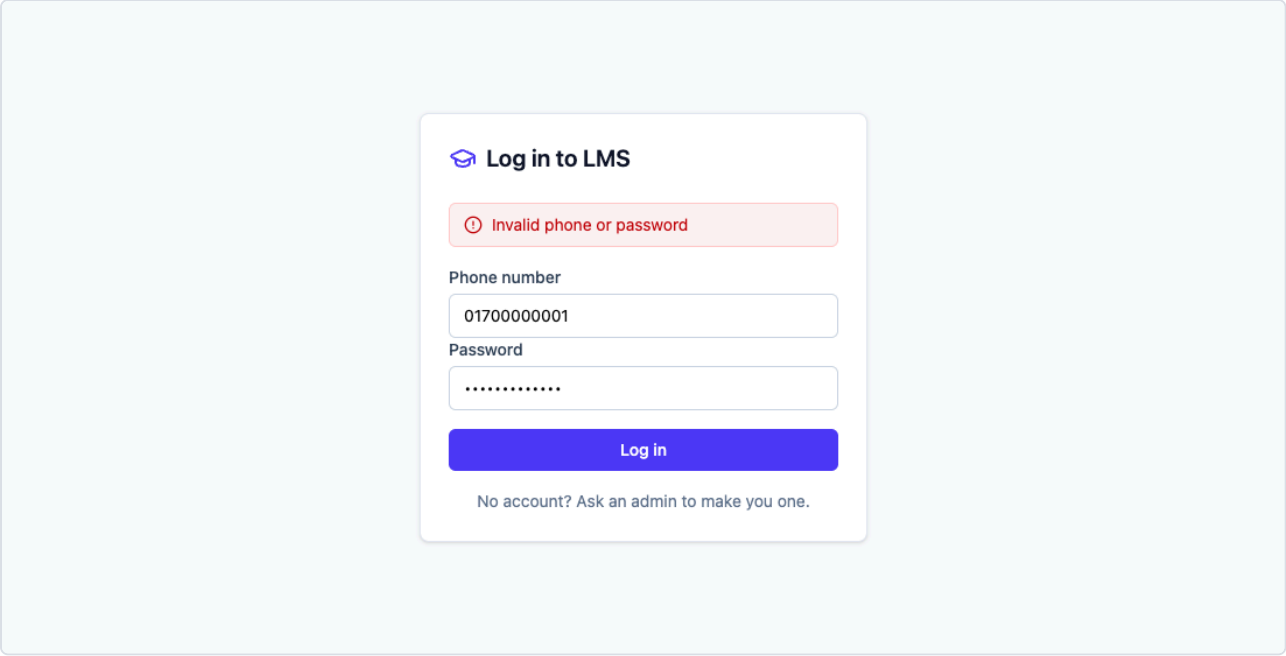

Figure 4.1 One message for two different failures. That is not laziness.

The view really does have two separate failure paths — no Profile with that phone number, and a Profile whose password did not match — and it deliberately answers both with the same sentence and the same status code:

backend/backend/views.py

```
        try:
            profile = Profile.objects.get(phone=phone)
            user = profile.user
        except Profile.DoesNotExist:
            return Response({'error': 'Invalid phone or password'},
status=status.HTTP_400_BAD_REQUEST)

        if not user.check_password(password):
            return Response({'error': 'Invalid phone or password'},
status=status.HTTP_400_BAD_REQUEST)
```

If the messages differed — "no such phone number" versus "wrong password" — anyone could feed the endpoint a list of phone numbers and learn which ones have accounts, without guessing a single password. That is called **account enumeration**, and it turns a login form into a directory of your users. One vague message closes it.

The same reasoning shows up again in Chapter 9, where the forgotten-password endpoint answers "if an account exists with this email…" whether or not it does.

See it for yourself

Django's shell is the fastest way to make this concrete. From `backend/`, with the virtual environment active:

```
python manage.py shell
```

```
from django.contrib.auth.hashers import make_password, check_password

h1 = make_password("opensesame")
h2 = make_password("opensesame")

print(h1)
print(h2)
print(h1 == h2)                     # False – different salts

print(check_password("opensesame", h1))   # True
print(check_password("opensesame", h2))   # True
print(check_password("Opensesame", h1))   # False
```

Then look at a real account:

```
from django.contrib.auth import get_user_model
User = get_user_model()

u = User.objects.first()
print(u.username, u.password)        # the stored hash, not a password
```

> **CHECKPOINT** You have hashed one word twice and got two different strings, and confirmed both verify against the original. If you can explain to somebody else why that is not a contradiction, this chapter is done.

What you have now

You know that passwords are hashed rather than stored, what the four `$`-separated parts of a Django hash mean, why a salt exists and why it is not a secret, and which three methods do the work: `create_user`, `set_password` plus `save`, and `check_password`. You know what the four validators refuse and where they are called, and why one deliberately vague error message is better security than two accurate ones.

Next: the login view, line by line.

CHAPTER 5

Login, Line by Line

Everything so far has been preparation. This is the chapter where somebody actually signs in. The whole thing is thirty lines of Python and one small React form, and by the end you should be able to recite the order of events without looking.

The login screen itself

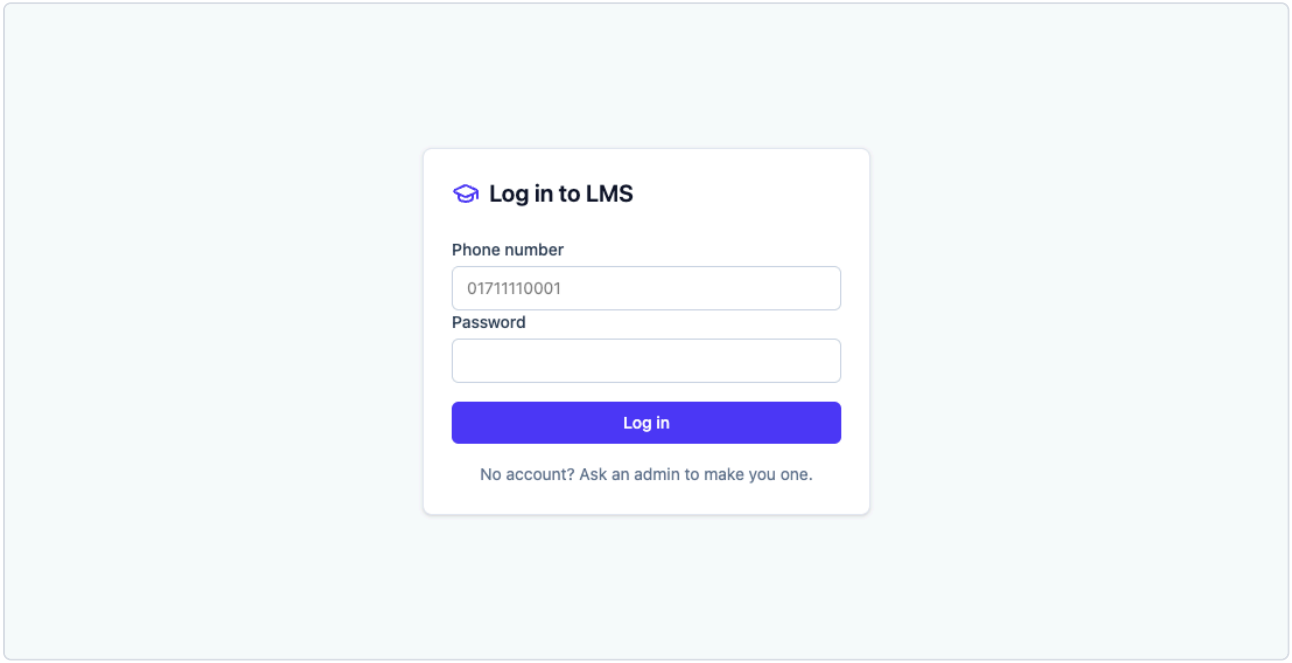

Figure 5.1 Two fields and a button. Note the line underneath: there is no sign-up link, because of Chapter 3.

The form has one job — collect two strings and hand them to `logIn`:

frontend/src/pages/Login.jsx

```
  async function handleSubmit(event) {
    event.preventDefault();
    setError("");
    setIsSaving(true);

    try {
      // The backend looks you up by phone number, not by username.
      await logIn(phone, password);
      navigate(goBackTo, { replace: true });
    } catch (problem) {
      setError(problem.message);
    } finally {
      setIsSaving(false);
    }
  }
```

`event.preventDefault()` stops the browser doing what HTML forms normally do, which is reload the whole page. This app handles the submission itself in JavaScript, so the page must stay put. Every React form in the project starts with that line.

`goBackTo` is a small courtesy explained in Chapter 7: if you were sent here from a page you were trying to reach, you get sent back to it rather than to the dashboard.

The serializer that only checks shape

backend/backend/serializers.py

```
class LoginSerializer(serializers.Serializer):
    phone = serializers.CharField(required=True)
    password = serializers.CharField(required=True, write_only=True)
```

Compare this with `RegisterSerializer`. That one was a `ModelSerializer`, tied to the `User` table, and it created rows. This is a plain `Serializer`, tied to nothing, and it creates nothing. Logging in does not write anything down, so there is no model to point at.

Its entire job is to answer one question: *did the request contain both fields, as text?* Nothing here checks whether the phone number exists or the password is right. That is the view's job, and keeping the two apart is why the code reads cleanly.

The view

backend/backend/views.py

```
class LoginView(APIView):
    """Login view using phone and password."""

    permission_classes = [AllowAny]

    def post(self, request):
        serializer = LoginSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        phone = serializer.validated_data['phone']
        password = serializer.validated_data['password']

        try:
            profile = Profile.objects.get(phone=phone)
            user = profile.user
        except Profile.DoesNotExist:
            return Response({'error': 'Invalid phone or password'},
status=status.HTTP_400_BAD_REQUEST)

        if not user.check_password(password):
            return Response({'error': 'Invalid phone or password'},
status=status.HTTP_400_BAD_REQUEST)

        tokens = get_tokens_for_user(user)
        return Response({
            'message': 'Login successful',
            'user_id': user.id,
            'username': user.username,
            # The frontend uses this to decide which buttons to show. It is
            # not what enforces anything: the permission classes do that.
            'role': role_of(user),
            'tokens': tokens
        })
```

Two things about the class line before the body. `APIView` is the hand-written kind of DRF view: you write the `post` method yourself, unlike `RegisterView` which inherited its behaviour. And `permission_classes = [AllowAny]` is essential, because of this in `settings.py`:

backend/lms/settings.py

```
    "DEFAULT_PERMISSION_CLASSES": (
        "rest_framework.permissions.IsAuthenticated",
    ),
```

Every endpoint in this project demands a signed-in caller *by default*. Without `AllowAny`, the login endpoint would require you to be logged in before you could log in. Say that out loud once and you will never forget to put it on a public endpoint again.

Step 1: is the request even the right shape?

```
        serializer = LoginSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        phone = serializer.validated_data['phone']
        password = serializer.validated_data['password']
```

`request.data` is the JSON that arrived, already turned into a Python dictionary by DRF. `is_valid(raise_exception=True)` means "check it, and if it is wrong, stop right here and answer 400 with the reasons" — no `if` statement needed, because the exception ends the method.

Send `{"phone": "0171..."}` with no password and you get:

```
{"password": ["This field is required."]}
```

Then the two values are read out of `validated_data`, not out of `request.data`. That distinction matters: `validated_data` holds the cleaned, type-checked versions, and reading from it is the habit that keeps unchecked input out of your logic.

Step 2: find the account by phone

```
        try:
            profile = Profile.objects.get(phone=phone)
            user = profile.user
        except Profile.DoesNotExist:
            return Response({'error': 'Invalid phone or password'},
status=status.HTTP_400_BAD_REQUEST)
```

Here is the chapter's key sentence, and the one that trips up everybody who has used another Django app: **the lookup starts in `Profile`, not in `User`.** The phone number lives on the Profile, so that is where the search happens, and `profile.user` then follows the one-to-one link from Chapter 2 to the account itself.

`.get()` means "exactly one row, or raise". It raises `Profile.DoesNotExist` when there is none — caught here — and `MultipleObjectsReturned` when there are several, which cannot happen because `phone` is `unique=True`. That is the uniqueness rule from Chapter 2 quietly paying for itself.

Step 3: check the password

```
        if not user.check_password(password):
            return Response({'error': 'Invalid phone or password'},
status=status.HTTP_400_BAD_REQUEST)
```

One line, and Chapter 4 explained all of it: the stored hash is taken apart, the typed password is hashed the same way, the two are compared. Same message as step 2, on purpose.

Everything below this line runs only for someone who has proved who they are. This is the exact moment authentication succeeds.

Step 4: mint the tokens

backend/backend/views.py

```
def get_tokens_for_user(user):
    """Helper to get JWT tokens for a user."""
    refresh = RefreshToken.for_user(user)
    return {
        'refresh': str(refresh),
        'access': str(refresh.access_token),
    }
```

`RefreshToken.for_user(user)` comes from the `djangorestframework-simplejwt` package. It builds a refresh token carrying this user's id, then `refresh.access_token` derives a short-lived access token from it. Both are turned into strings, because what travels over the network is text.

Nothing is written to any table. That is worth pausing on: at the end of a successful login this app has not recorded that you logged in. The proof went out the door with you. Chapter 6 is entirely about what is inside those strings.

Step 5: the answer, and its nested shape

```
{
  "message": "Login successful",
  "user_id": 7,
  "username": "nadia",
  "role": "teacher",
  "tokens": {
    "refresh":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2tlbl90eXBlIjoicmVmcmVzaCIsImV4cCI6MTc...",
    "access":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2tlbl90eXBlIjoiYWNjZXNzIiwiZXhwIjoxNzU..."
  }
}
```

Three things to notice.

The tokens are nested. They are under `tokens`, so the frontend reads `data.tokens.access`. Most JWT tutorials answer with `access` at the top level, which means most copied code reads `data.access` and gets `undefined`. The comment in `AuthContext.jsx` exists because of exactly this:

frontend/src/AuthContext.jsx

```
    // The tokens are nested. data.access does not exist.
    const nextUser = {
      id: data.user_id,
      username: data.username,
      role: data.role,
    };
    saveSession(data.tokens.access, nextUser);
```

The role is included. Not because the frontend is trusted with it — the note in the view says so plainly — but so the sidebar can hide links a student cannot use. The server checks the role again on every request regardless. Chapter 8.

The refresh token is sent but never used. This backend has no refresh endpoint. The frontend stores only the access token and drops the refresh one on the floor. Chapter 6 explains what it would have been for.

The sequence on one page

CHECKPOINT Open the browser's developer tools, go to the Network tab, and log in. Click the `login/` request. Under **Payload** you will see the two fields you typed; under **Response**, the JSON above with two long token strings. You are looking at the entire authentication step, in the raw.

What you have now

You can trace a login from the form to the response: shape check, lookup by phone in the `Profile` table, password check against the hash, two tokens signed and returned. You know why `AllowAny` is compulsory here, why both failures share one message, and why the tokens arrive nested under `tokens`.

Next: what is actually inside those strings.

CHAPTER 6

The Token Itself: What a JWT Is

Login handed the browser a long line of gibberish. It is not gibberish. This chapter takes one apart with nothing but a text editor, and then explains the two properties — it is readable by anyone, and it cannot be taken back — that shape half the decisions in this project.

A token is a signed note

Back to the office building. The session approach from Chapter 1 gives you a numbered claim ticket and keeps a ledger at the desk. The token approach gives you a note that reads "*the bearer of this note is employee 7, valid until 6pm*", signed by the manager, and keeps no ledger at all.

Anyone can read the note. Nobody can alter it, because the signature would stop matching. Anyone holding it is treated as employee 7, which is why the word in the HTTP header is literally **Bearer**.

JWT stands for JSON Web Token, said "jot". It is that note, written as JSON and signed.

The three parts

Here is a real access token from this project, wrapped over three lines so it fits on the page:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.eyJ0b2tlbl90eXBlIjoiYWNjZXNzIiwiZXhwIjoxNzg1ODY4ODU0LCJpYXQiOjE3ODU4NDAwNTQsImp0aSI6
ImQ4ZDA3YzJmNWI2YjRmZDk5ZGZkNTZmNTg5ODVhZmY3IiwidXNlcl9pZCI6N30
.N8vyuuXgIBYFPG5cCbQz27dPnTs7Bpy5QG9WUW_VQrY
```

Three parts separated by full stops: **header**, **payload**, **signature**. The first two are JSON that has been **base64url encoded** — a way of writing any text using only letters, digits, `-` and `_`, so it survives being put in a URL or a header. Encoding is not encryption. It is not even slightly secret; it is more like writing in capitals.

That is also why almost every JWT you will ever see starts with `eyJ`: those are the base64 letters for `{"`, the opening of any JSON object.

Decode one yourself

Take the first part, `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9`, and decode it:

```
{"alg":"HS256","typ":"JWT"}
```

The header says how the token was signed: HMAC with SHA-256. Nothing about you.

The middle part, decoded and laid out:

```
{
  "token_type": "access",
  "exp": 1785868854,
  "iat": 1785840054,
  "jti": "d8d07c2f5b6b4fd99dfd56f58985aff7",
  "user_id": 7
}
```

CLAIM	SHORT FOR	WHAT IT DOES HERE
`token_type`	—	`access` or `refresh`. Stops a refresh token being used where an access token is expected.
`exp`	expires	The moment it stops working, as seconds since 1 January 1970. Compare it with `iat`: exactly 28,800 apart, which is eight hours.
`iat`	issued at	When it was signed.
`jti`	JWT id	A unique id for this one token. It is what the blacklist in Chapter 9 keeps track of.
`user_id`	—	The `auth_user.id` from Chapter 2. This single number is how the server knows who you are.

These short names are not this project's invention; `exp`, `iat` and `jti` are part of the JWT standard, which is why libraries in every language understand them.

Note what is *not* in there: no password, no phone number, and no role. The server looks the role up fresh on every request, which is exactly why demoting someone in the Django admin takes effect immediately instead of when their token expires.

> **TRY IT** Decode a token of your own without any tools. In Python: `import base64, json`, then take the middle section, add `"=" * (-len(part) % 4)` to pad it, and run `json.loads(base64.urlsafe_b64decode(part))`. The website `jwt.io` does the same thing in a browser — but never paste a live production token into a website.

Not encrypted. Signed.

This deserves its own heading because it catches out even experienced developers.

A JWT is readable by anybody who has it. Encoding is not encryption. If you put a person's email address, salary or role in a token, everyone holding that token can read it, including the person themselves and anyone who steals it.

What the signature guarantees is not secrecy but **integrity**: nobody can change the contents. Try editing `"user_id": 7` to `"user_id": 1` and re-encoding, and the signature no longer matches the payload. The server rejects it, because the signature can only be produced by someone holding the secret key.

So the rule for what goes in a payload is simple: facts you would not mind the holder reading, and nothing you would mind them knowing.

The signature and SECRET_KEY

The third part is the result of running header + payload + a secret through HMAC-SHA256. The secret is this line:

backend/lms/settings.py

```
# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-@hg+%+(*w2m0jh=8$ksnpsoe+dv2jb+py3@1b(3vhohjb7cx8!'
```

Everything rests on that string. Anybody who knows it can sign a token claiming to be `user_id: 1`, and your server will believe it completely, because believing signatures is the entire mechanism. Django even names the auto-generated one `django-insecure-…` to nag you.

> **DO NOT SHIP THIS**
>
> A `SECRET_KEY` committed to Git is a real, exploitable vulnerability, not a style issue. On a real deployment it must be read from an environment variable and never appear in the repository. Chapter 12 lists this alongside the other things to fix before the app faces the internet.

Changing `SECRET_KEY` invalidates every token that was signed with the old one, so every user is logged out at once. That is worth knowing both as a consequence and as a blunt emergency tool.

Access and refresh

Login returned two tokens. The pattern they come from is standard, even though this project only half-uses it.

	ACCESS TOKEN	REFRESH TOKEN
Sent with	Every request, in the header	Only to the refresh endpoint
Lifetime here	8 hours	7 days
Normal lifetime	5–15 minutes	Days or weeks
If stolen	Damage is limited by the clock	Bad: it mints fresh access tokens
Used in this app	Yes, constantly	No. Received and discarded.

The idea is a trade. The access token travels everywhere, so it is exposed constantly and is therefore made short-lived. The refresh token barely travels, so it can be long-lived, and its only power is to produce new access tokens.

backend/lms/settings.py

```
# The library default access token lifetime is 5 minutes, which forces a
# re-login mid-task during development. There is no token refresh endpoint,
# so the access token is widened instead.
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(hours=8),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
    "AUTH_HEADER_TYPES": ("Bearer",),
}
```

The eight-hour decision

That comment is a design decision written down honestly, and it is worth reading as one.

The library default is five minutes. Without a refresh endpoint, five minutes means the app throws you back to the login screen in the middle of typing a lesson description. So the author widened the access token to eight hours — a working day — instead of building the refresh flow.

What that costs: a stolen access token is useful to a thief for up to eight hours instead of up to five minutes.

What it buys: an app that is usable for a day's work, and one less endpoint to build and explain.

For a teaching project on a laptop, that is a defensible trade. For a real system holding real student records, it is not, and the fix is the refresh endpoint, not an even longer lifetime. The habit worth taking from this: when you loosen a security setting, write down what you traded and why, the way this file does.

`"AUTH_HEADER_TYPES": ("Bearer",)` is the third line, and it is the one you will meet in practice. It says the header must read `Authorization: Bearer <token>`. Write `Token <token>` or `JWT <token>` — both are common in other frameworks — and this server treats you as a stranger.

The thing tokens are bad at

Here is the cost of the whole approach, and this app runs straight into it.

You cannot un-sign a signed note. The server keeps no list of valid tokens, so there is nothing to delete. Once an access token is out there, it works until `exp` passes, and nothing short of changing `SECRET_KEY` can stop it. Compare with sessions, where logging somebody out is one `DELETE`.

Three places in this project bump into that fact, and each handles it honestly rather than pretending:

- **Logging out** (Chapter 7): the frontend deletes its copy of the token. The token itself remains valid until it expires. There is no logout endpoint, because there is nothing useful for one to do.
- **Changing your password** (Chapter 9): refresh tokens are blacklisted, the access token cannot be, and so the app signs you out on the spot and makes you log back in.
- **Demoting someone**: this one is fine, and by design. The role is not in the token, so the change applies to the very next request.

The industry answer to all of this is short access tokens plus refresh. Which brings the chapter back to where it started: the eight-hour setting is the reason this app cannot do better than "delete your copy and hope".

What you have now

You can take a JWT apart by hand and say what each of the three parts is for. You know the five claims in this project's payload, that `user_id` is the whole of what identifies you, and that the role is deliberately absent. You know a token is signed rather than encrypted, what `SECRET_KEY` is guarding, why the access lifetime is eight hours here, and why handing out signed notes makes taking one back so hard.

Next: the browser side — where that token is kept and how it gets attached to every request.

CHAPTER 7

Staying Logged In: The Browser Half

The server's part is over. It handed out a signed note and forgot the whole thing. From here on, "being logged in" is something the browser does, and it is the browser's job to hold on to that note and produce it every time it knocks.

Three files, one job

FILE	RESPONSIBILITY
`src/api.js`	Every call to Django. Stores the token, attaches it, turns error responses into readable messages.
`src/AuthContext.jsx`	Holds "am I logged in, and who am I" so any component can ask.
`src/components/ProtectedRoute.jsx`	Stands in front of pages that need a signed-in visitor.

Plus a two-line helper, `src/auth.js`, holding the context object and the `useAuth` hook. It exists separately for a mundane reason worth knowing:

frontend/src/auth.js

```
// The context and its hook live apart from AuthContext.jsx so that file only
// exports a component. Vite's fast refresh gives up on any file that mixes
// components with other exports.
export const AuthContext = createContext(null);
```

localStorage: the browser's little notebook

The token has to survive you pressing F5. A JavaScript variable does not: a page reload wipes every variable the app had. So it goes in **localStorage**, a small store of text the browser keeps per website, permanently, until code or the user clears it.

frontend/src/api.js

```
const TOKEN_KEY = "lms_access_token";
const USER_KEY = "lms_user";

export function getToken() {
  return localStorage.getItem(TOKEN_KEY);
}

export function saveSession(token, user) {
  localStorage.setItem(TOKEN_KEY, token);
  localStorage.setItem(USER_KEY, JSON.stringify(user));
}

export function getSavedUser() {
  const raw = localStorage.getItem(USER_KEY);
  if (!raw) {
    return null;
  }
  try {
    return JSON.parse(raw);
  } catch {
    return null;
  }
}

export function clearSession() {
  localStorage.removeItem(TOKEN_KEY);
  localStorage.removeItem(USER_KEY);
}
```

Two entries under two fixed keys. localStorage only holds strings, which is why the user object is put through `JSON.stringify` going in and `JSON.parse` coming out, and why the parse is wrapped in `try` — if something ever writes rubbish under that key, the app should treat it as "not logged in" rather than crash on startup.

> **SEE IT** Developer tools → Application (or Storage) → Local Storage → your site. Two rows: `lms_access_token` holding the long string from Chapter 6, and `lms_user` holding something like `{"id":7,"username":"nadia","role":"teacher"}`. Delete the token row and reload: you are on the login screen. That is all logging out is.

Note what the saved user does *not* contain: no email, no phone. Just enough to greet you and draw the right menu. The full details are fetched from `/api/profile/` when the profile page opens.

Attaching the token to every call

Here is the heart of the file, and the answer to "how does the server know it is me":

frontend/src/api.js

```
async function request(path, options = {}) {
  const { auth = true, ...rest } = options;

  const headers = { ...(rest.headers ?? {}) };
  if (rest.body !== undefined) {
    headers["Content-Type"] = "application/json";
  }

  const token = getToken();
  if (auth && token) {
    headers.Authorization = `Bearer ${token}`;
  }

  let response;
  try {
    response = await fetch(BASE_URL + path, { ...rest, headers });
  } catch {
    throw new Error(
      "Could not reach the server. Is Django running on http://127.0.0.1:8001?",
    );
  }
  ...
```

Every call in the app goes through this one function, which is exactly why it can guarantee the header is there. Three details:

- `auth = true` is the default, so calls are authenticated unless they say otherwise. Only three calls opt out: login, and the two halves of the password reset flow. Defaulting to *secure* and opting out deliberately is the same instinct as `DEFAULT_PERMISSION_CLASSES` on the server.
- The header is spelled `Authorization: Bearer <token>`, matching `AUTH_HEADER_TYPES` from Chapter 6. Note the space and the capital B.
- `Content-Type: application/json` is only set when there is a body, because a GET with no body has nothing to describe.

The `catch` around `fetch` deserves a note. It fires when the request never reached a server at all — Django not running, wrong port, no network. That is different from a request that arrives and is refused, which is a normal response with a status code.

WHICH PORT

Both that message and the base URL a few lines above name the same port, and it is not Django's default:

```
const BASE_URL = import.meta.env.VITE_API_URL ??
"http://127.0.0.1:8001/api";
```

Django's own default is 8000, so `python manage.py runserver` on its own leaves the frontend calling a port with nothing behind it. That is why the README says `python manage.py runserver 8001`. The pairing to keep is the port Django listens on and the port `api.js` calls: they have to be the same number. If you want a different one, run Django there and create `frontend/.env` containing `VITE_API_URL=http://127.0.0.1:<that port>/api`. A mismatch here is the single most common "nothing works" on a first run.

AuthProvider, and what context is for

Lots of components need to know who is signed in: the sidebar draws the menu from it, the dashboard greets you by name, every page hides buttons based on the role. Passing that down through every layer of components by hand would be miserable. React's **context** is the answer — a value provided high up in the tree and read by any component below, however deep.

frontend/src/main.jsx

```
createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <AuthProvider>
      <App />
    </AuthProvider>
  </BrowserRouter>,
);
```

`AuthProvider` wraps the entire app, so everything inside can call `useAuth()` and get four things: `user`, `isLoggedIn`, `logIn` and `logOut`.

frontend/src/AuthContext.jsx

```
export function AuthProvider({ children }) {
  // The token is read back out of localStorage on the first render so a page
  // refresh does not log you out.
  const [user, setUser] = useState(() => getSavedUser());
  const [isLoggedIn, setIsLoggedIn] = useState(() => Boolean(getToken()));
```

Logging in, from the frontend's side

frontend/src/AuthContext.jsx

```
  async function logIn(phone, password) {
    const data = await loginRequest(phone, password);

    // The tokens are nested. data.access does not exist.
    const nextUser = {
      id: data.user_id,
      username: data.username,
      role: data.role,
    };
    saveSession(data.tokens.access, nextUser);

    setUser(nextUser);
    setIsLoggedIn(true);
  }
```

Five lines that close the loop from Chapter 5: call the endpoint, pick the three useful fields out of the answer, write the access token and the user into localStorage, and update React's state so the interface redraws itself as a signed-in app.

Both are needed. localStorage survives a reload but React does not watch it; React state redraws the screen but vanishes on reload. Writing to one without the other gives you the two classic bugs: an app that still looks signed out until you refresh, or one that looks signed in until you refresh.

Surviving a page refresh

The two `useState` calls above take a function — `() => getSavedUser()` — rather than a value. React calls it once, on the very first render, to work out the starting state. So the app boots by asking localStorage "was somebody signed in here?" and starts in the right state immediately, with no flicker through a logged-out screen.

There is one more wrinkle, and the comment explains it better than a paraphrase:

frontend/src/AuthContext.jsx

```
// A session saved before roles existed has no role on it. Rather than treat
// that account as having no permissions, ask /profile/ what it is.
useEffect(() => {
  if (!isLoggedIn || user?.role) {
    return;
  }

  let stillMounted = true;

  async function fillInTheRole() {
    try {
      const data = await fetchProfile();
      if (!stillMounted) {
        return;
      }
      const nextUser = {
        id: data.user.user_id,
        username: data.user.username,
        role: data.user.role,
      };
      saveSession(getToken(), nextUser);
      setUser(nextUser);
    } catch {
      // A dead token is already handled by api.js; nothing to add here.
    }
  }

  fillInTheRole();
  return () => {
    stillMounted = false;
  };
}, [isLoggedIn, user?.role]);
```

Roles were added to this app after some people had already logged in. Their saved `lms_user` has no `role`, and treating that as "no permissions" would have locked working accounts out of their own buttons. So the app notices the gap and asks the server. This is what a migration looks like on the browser side: old data, still in people's browsers, where you cannot reach it except by handling it.

The `stillMounted` flag is standard practice for any `useEffect` that awaits something. If the component disappears while the request is in flight, the answer is thrown away instead of setting state on a component that is no longer there.

ProtectedRoute: the locked door in the router

frontend/src/components/ProtectedRoute.jsx

```jsx
// `role`, when given, is the one role allowed through. Hiding a link in the
// sidebar is not enough on its own — somebody can always type the URL.
export default function ProtectedRoute({ role, children }) {
  const { isLoggedIn, user } = useAuth();
  const location = useLocation();

  if (!isLoggedIn) {
    // `state` remembers where you were headed so the login page can send you
    // back there instead of always dropping you on the dashboard.
    return <Navigate to="/login" replace state={{ from: location.pathname }} />;
  }

  // The role arrives a moment after the token when a saved session is being
  // topped up from /profile/. Waiting avoids bouncing the user by mistake.
  if (role && !user?.role) {
    return null;
  }

  if (role && user.role !== role) {
    return <Navigate to="/" replace />;
  }

  return children;
}
```

Three checks, in order. Not signed in? Off to `/login`, with a note of where you were going — that note is the `goBackTo` from Chapter 5. A role is required but not known yet? Draw nothing for a moment rather than guess wrong, because of the `/profile/` top-up above. Signed in with the wrong role? Back to the dashboard.

In `App.jsx` it appears twice: once around the whole sidebar layout, so every inside page needs a token, and once more, tighter, around the Accounts page:

frontend/src/App.jsx

```
        {/* Admins only, and the API refuses everyone else regardless. */}
        <Route
          path="/accounts"
          element={
            <ProtectedRoute role="admin">
              <Accounts />
            </ProtectedRoute>
          }
        />
```

Read that comment carefully, because it is the most important sentence in the chapter. **ProtectedRoute is a convenience, not a security control.** It runs in the browser, on code the visitor could edit. It exists so people are not shown doors that will not open for them. The thing that actually stops a student creating accounts is `IsAdmin` on the server, which is Chapter 8.

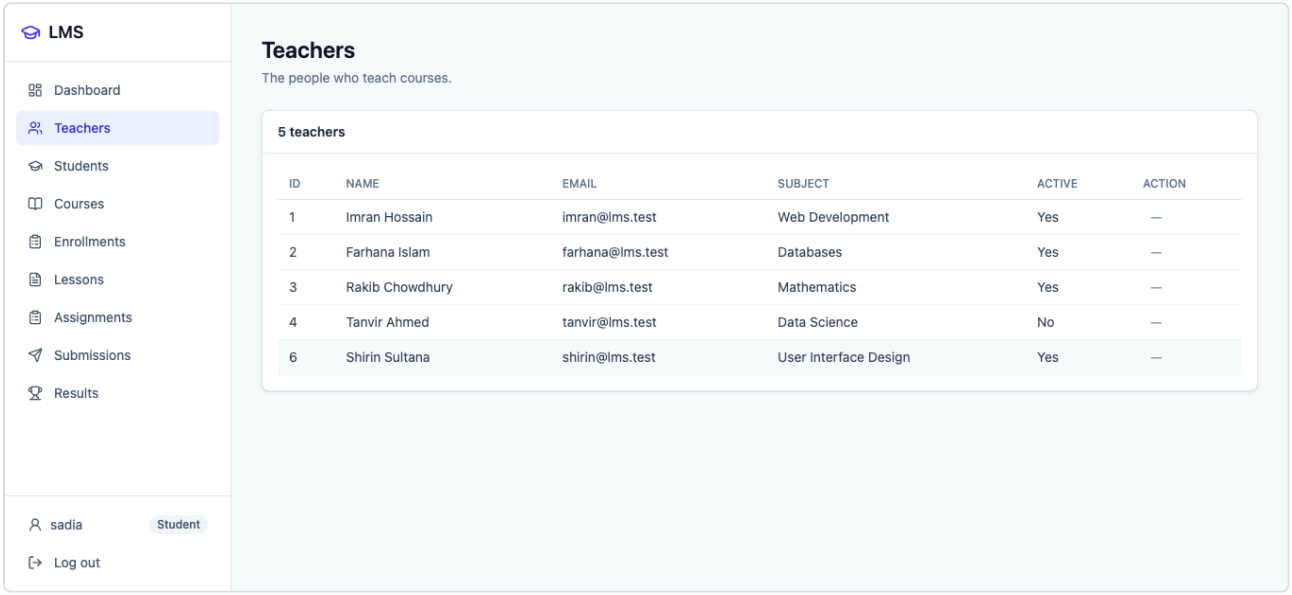

Figure 7.1 The sidebar for a student. Reading is open to any signed-in account, so every resource page is listed; the Accounts link is not drawn at all.

When the token dies mid-session

Eight hours pass, or the token is altered, and the next call comes back 401. There is no refresh endpoint to fall back on, so:

frontend/src/api.js

```
// There is no refresh endpoint on this backend. When the access token is
// rejected the only thing left to do is drop the session and let the router
// send the user back to /login.
function handleExpiredToken() {
  clearSession();
  window.dispatchEvent(new Event("lms:unauthorised"));
}

  if (response.status === 401 && auth) {
    handleExpiredToken();
    throw new Error("Your session has expired. Please log in again.");
  }
```

The problem is that `api.js` is a plain module, not a React component, so it cannot call `setIsLoggedIn`. It solves that by shouting into the room: it fires a custom browser event named `lms:unauthorised`, and the provider listens for it.

frontend/src/AuthContext.jsx

```
  // api.js fires this event when the backend rejects the token. There is no
  // refresh endpoint, so the session is simply dropped.
  useEffect(() => {
    function handleUnauthorised() {
      setUser(null);
      setIsLoggedIn(false);
    }

    window.addEventListener("lms:unauthorised", handleUnauthorised);
    return () =>
      window.removeEventListener("lms:unauthorised", handleUnauthorised);
  }, []);
```

State flips to signed-out, ProtectedRoute notices on the next render, and you land on the login page. The `return` inside the effect removes the listener again when the provider unmounts — the cleanup half of `useEffect`, and the thing beginners most often leave out.

Logging out of a stateless server

frontend/src/AuthContext.jsx

```
// There is no logout endpoint. Discarding the token is all the frontend
// can do; it stays valid on the server until it expires.
function logOut() {
  clearSession();
  setUser(null);
  setIsLoggedIn(false);
}
```

No network call at all. The Log out button in the sidebar deletes two rows from localStorage and flips two pieces of state.

Be clear about what that means: **the token is not cancelled.** If someone copied it before you pressed the button, it keeps working until `exp`. That is Chapter 6's cost, arriving on schedule. The honest summary is that logging out here protects you from the next person to use your computer, and from nobody else.

> **CHECKPOINT** Log in. Copy the value of `lms_access_token` out of localStorage into a text file. Press Log out, confirm you are back at the login screen, then paste the token back in under the same key and reload the page. You are signed in again, without a password. Nothing is broken; that is what "the server keeps no ledger" means, and it is why the button cannot promise more than it does.

What you have now

You know where the token lives between requests, how one wrapper function guarantees the header is attached, and how React context spreads "who is signed in" through the app. You know why a page refresh does not log you out, what ProtectedRoute does and why it is not security, how a dead token gets from `api.js` back to the router through a custom event, and what logging out really consists of.

Next: what the server does with that header, and how being *signed in* becomes being *allowed*.

CHAPTER 8

What Being Logged In Gets You

The badge is in your hand. This chapter is about the card readers on the doors: how Django turns a header into a person, and how each endpoint decides whether that person may do what they are asking.

Authentication hands over to authorisation

Chapter 1 put it in a sentence: authentication is *who are you*, authorisation is *what may you do*. Everything up to now has been the first. From here it is the second, and the switch happens in the middle of every single request:

```
request arrives
      |
      v
[authentication]   read the Authorization header,
                   check the signature, load user 7
      |
      v
[authorisation]    what is user 7's role?
                   does this endpoint allow that role
                   to do this method?
      |
      v
the view runs, or 401 / 403 comes back
```

How DRF turns a token into request.user

You never wrote code to read the `Authorization` header. One setting arranged it:

backend/lms/settings.py

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": (
        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ),
    ...
}
```

Before any view runs, `JWTAuthentication` does this, in order:

1. Look for an `Authorization` header. Nothing there? Stop; the caller is anonymous.

2. Check it starts with `Bearer`, per `AUTH_HEADER_TYPES`. Take the rest as the token.
3. Verify the signature with `SECRET_KEY`. Altered in any way? Reject.
4. Check `exp` against the clock. Expired? Reject.
5. Read `user_id` out of the payload, load that row from `auth_user`, and put it on the request.

After which every view can simply write `request.user` and get a real `User` object. When nobody is signed in, `request.user` is Django's `AnonymousUser`, whose `is_authenticated` is `False` — which is why the permission code checks that flag rather than testing for `None`.

> **NOTE** Step 5 is a database read on every request. That is the price of not putting the role in the token: the account is loaded fresh each time, so a change made in the Django admin takes effect on the very next call rather than in eight hours.

Locked by default

backend/lms/settings.py

```
    # Without this, DRF's default is AllowAny, so any view added without its
    # own permission_classes would be public. Failing closed means a new view
    # is locked until somebody decides otherwise; the two views that really
    # are public (register, login) say so with AllowAny.
    "DEFAULT_PERMISSION_CLASSES": (
        "rest_framework.permissions.IsAuthenticated",
    ),
```

This is one line and it is the best security decision in the project. Out of the box, DRF allows anyone; a view added in a hurry, with no `permission_classes`, would be open to the world and nothing would warn you. Flipping the default means the forgetful case is the *locked* case.

The jargon is **fail closed**: when in doubt, refuse. Every endpoint in this project now starts from "signed-in accounts only" and opens up deliberately from there. Three do: `LoginView` and the two password-reset views, all marked `AllowAny`. One goes the other way and gets stricter: `RegisterView`, with `IsAdmin`.

The comment says register is public; it is not, and that is the one place the comments in this project have drifted behind the code. `RegisterView` carries `permission_classes = [IsAdmin]`, as Chapter 3 showed.

role_of: one function, three answers

All the role logic starts in a single helper, seen briefly in Chapter 2 and worth reading once more now that you know what `request.user` is:

backend/backend/permissions.py

```python
def role_of(user):
    """The caller's role, or None when nobody is signed in.

    Two accounts have no Profile row: a superuser made with `createsuperuser`,
    and anything created outside the register endpoint. A superuser is treated
    as an admin; anyone else without a Profile gets the least privilege we
    have rather than an exception.
    """
    if not user or not user.is_authenticated:
        return None

    if user.is_superuser:
        return Profile.ADMIN

    profile = Profile.objects.filter(user=user).only("role").first()
    if profile is None:
        return Profile.STUDENT

    return profile.role
```

Four exits, and each is a decision:

SITUATION	ANSWER	WHY
Nobody signed in	`None`	`None` matches no role, so every role test fails. Fail closed again.
Superuser	`"admin"`	So `createsuperuser` is always a way back in, even with no Profile.
Signed in, no Profile	`"student"`	Least privilege beats crashing. A missing row is not a promotion.
Normal account	The stored role	The ordinary case.

`.only("role")` asks the database for that one column instead of the whole row. A small thing, but this runs on every write request in the app.

The four permission classes

A **permission class** is an object with a `has_permission(self, request, view)` method that returns `True` or `False`. DRF calls it before the view runs. That is the entire interface.

IsAdmin

backend/backend/permissions.py

```python
class IsAdmin(BasePermission):
    """Admins only, for every method including GET.

    Used on registration: accounts are handed out by an admin, not claimed by
    whoever finds the URL.
    """

    message = "Only an admin can create accounts."

    def has_permission(self, request, view):
        return role_of(request.user) == Profile.ADMIN
```

The strictest of the four, and the only one that also blocks reading. `message` is what the caller sees in the refusal, which is why the errors in this app read like sentences rather than like "Permission denied".

RoleWritePermission, the shared base

backend/backend/permissions.py

```python
class RoleWritePermission(BasePermission):
    """Signed-in accounts can read. Only `roles_that_may_write` can write."""

    roles_that_may_write = ()
    message = "Your role does not allow this change."

    def has_permission(self, request, view):
        if not request.user or not request.user.is_authenticated:
            return False

        if request.method in SAFE_METHODS:
            return True

        return role_of(request.user) in self.roles_that_may_write
```

The pattern here is the one to remember: **reading is open to anyone signed in; writing depends on your role.**

`SAFE_METHODS` is DRF's tuple `("GET", "HEAD", "OPTIONS")` — the methods that only look at data. Anything else (POST, PUT, PATCH, DELETE) changes something, and falls through to the role check.

The class is never used directly. Two subclasses fill in the blank:

backend/backend/permissions.py

```python
class AdminWrites(RoleWritePermission):
    """Teacher and Student records: the register of who is in the school."""

    roles_that_may_write = (Profile.ADMIN,)
    message = "Only an admin can change teacher and student records."


class TeachingStaffWrites(RoleWritePermission):
    """Courses, lessons, assignments, enrollments and results."""

    roles_that_may_write = (Profile.ADMIN, Profile.TEACHER)
    message = "Only a teacher or an admin can change course material and grades."
```

SubmissionWrites, and the one exception

backend/backend/permissions.py

```python
class SubmissionWrites(RoleWritePermission):
    """Same as above, except a student may hand work in.

    A student cannot edit or delete a submission, because nothing links a
    Student row to a login account — so the server has no way to tell whose
    work it is. Handing in is the one write that is safe without that link.
    """

    roles_that_may_write = (Profile.ADMIN, Profile.TEACHER)
    message = "Students can hand work in, but cannot change or remove a submission."

    def has_permission(self, request, view):
        if (
            request.method == "POST"
            and request.user
            and request.user.is_authenticated
            and role_of(request.user) == Profile.STUDENT
        ):
            return True

        return super().has_permission(request, view)
```

One extra rule bolted onto the base: a signed-in student may POST, and only POST. Everything else falls through to `super()`, which is the ordinary teacher-or-admin check.

The docstring explains the limit honestly, and it is the most interesting sentence in the file. The `Student` table in this app is a list of people in the school; it has no link to a login account. So when a student POSTs a submission, the server cannot verify that the `student` id in the body is *them*.

Allowing edits or deletes on top of that would let any student rewrite anyone's work. Allowing the create is the largest amount of trust the data model can currently support. Chapter 12 comes back to this.

Attaching a class to an endpoint is one line:

backend/backend/views.py

```
class TeacherListCreateView(generics.ListCreateAPIView):
    queryset = Teacher.objects.all()
    serializer_class = TeacherSerializer
    permission_classes = [AdminWrites]
```

401 versus 403

These two numbers get muddled constantly, and telling them apart is the fastest debugging skill in this whole subject.

	401 UNAUTHORIZED	403 FORBIDDEN
Means	I do not know who you are	I know who you are, and no
Cause	No token, malformed token, bad signature, expired	Valid token, wrong role for this action
Body	`{"detail": "Authentication credentials were not provided."}`	`{"detail": "Only an admin can create accounts."}`
Fix	Log in again, or attach the header properly	Nothing you can do. Ask an admin.
The app's response	Clears the session, back to `/login`	Shows the message in a red banner, stays put

The names are historical and unhelpful — 401 "Unauthorized" is the authentication failure and 403 "Forbidden" is the authorisation failure, which is the opposite of what the words suggest. Learn the meanings, ignore the labels.

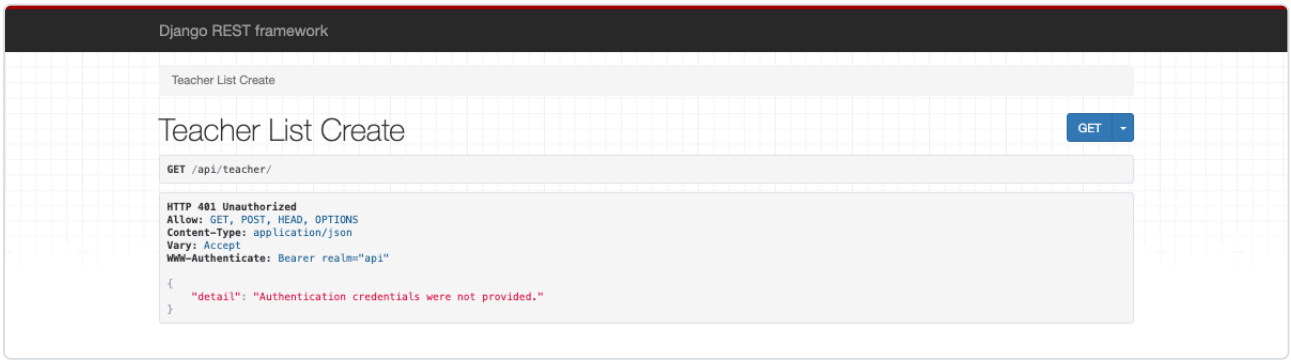

Figure 8.1 A plain browser tab cannot attach an `Authorization` header, so `/api/teacher/` answers 401 rather than the list.

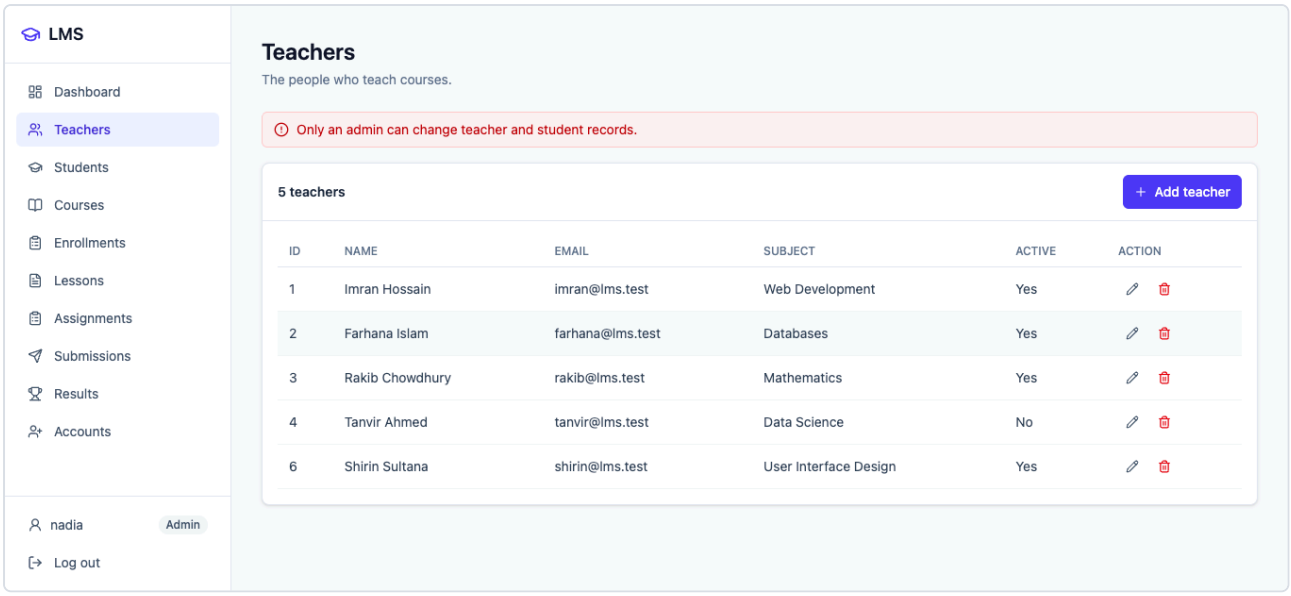

Figure 8.2 A 403 as the user sees it: the permission class's own `message`, straight through to the red banner.

The permission table

Read straight off the `permission_classes` lines in `views.py`:

ENDPOINT	CLASS	ADMIN	TEACHER	STUDENT
`/api/register/`	IsAdmin	All	Nothing	Nothing
`/api/login/`	AllowAny	Open to everyone, signed in or not		
`/api/profile/`	IsAuthenticated	Your own account only, whatever your role		
`/api/change-password/`	IsAuthenticated	Your own password only		
`/api/password-reset/`	AllowAny	Open — you are locked out when you need it		
`/api/teacher/`, `/api/student/`	AdminWrites	Read + write	Read	Read
`/api/course/`, `/api/lesson/`, `/api/assignment/`, `/api/enrollment/`, `/api/results/`	TeachingStaffWrites	Read + write	Read + write	Read
`/api/submission/`	SubmissionWrites	Read + write	Read + write	Read + create only

Two things stand out. Reading is open to every signed-in account, everywhere — a student can list teachers and courses. And a student's only write in the entire API is handing in a submission.

The frontend copy is cosmetic

The React app keeps its own copy of that table, and says so in the first three lines:

frontend/src/permissions.js

```
// A copy of the rules in backend/permissions.py, used only to decide which
// buttons are worth showing. The server is what actually enforces them, so a
// mistake here is a cosmetic bug, not a hole. Keep the two in step.

export const ADMIN = "admin";
export const TEACHER = "teacher";
export const STUDENT = "student";

const WRITERS = {
  teacher: [ADMIN],
  student: [ADMIN],
  course: [ADMIN, TEACHER],
  enrollment: [ADMIN, TEACHER],
  lesson: [ADMIN, TEACHER],
  assignment: [ADMIN, TEACHER],
  submission: [ADMIN, TEACHER],
  results: [ADMIN, TEACHER],
};

// May this role change or remove rows of this kind?
export function canWrite(role, resource) {
  return (WRITERS[resource] ?? []).includes(role);
}

// May this role add a new row? Same as canWrite everywhere except submissions,
// which a student is allowed to hand in but not to edit afterwards.
export function canCreate(role, resource) {
  if (resource === "submission" && role === STUDENT) {
    return true;
  }
  return canWrite(role, resource);
}
```

Duplicating rules is normally a mistake, so be precise about why it is right here. The two copies do different jobs. The server's copy is a **control**: it decides what happens. The browser's copy is a **hint**: it decides what is worth drawing. Showing a student an Add button that will always fail is a bad interface, and hiding it is a courtesy, nothing more.

The failure modes are correspondingly different. Get the server copy wrong and you have a security hole. Get the browser copy wrong and somebody sees a button that gives them a red banner. The comment's last three words — "keep the two in step" — are the maintenance cost of the arrangement,

and they are the price of not shipping a security check to the browser.

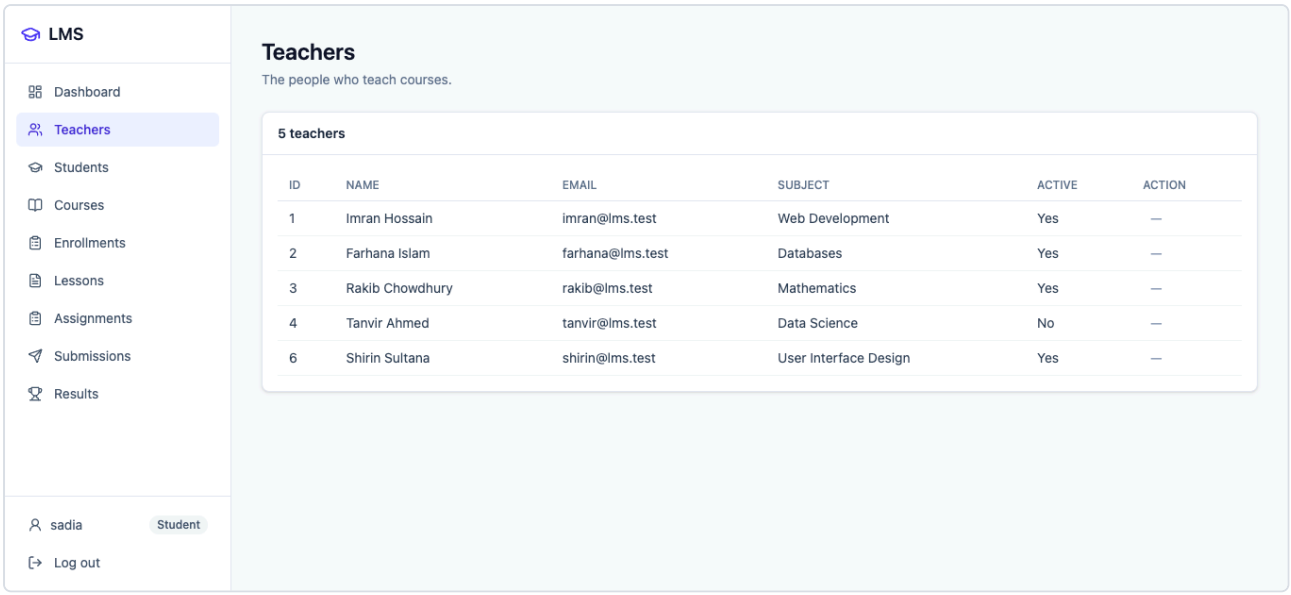

Figure 8.3 The Teachers page as a student. The list loads, because reading is open. No Add button, and dashes where the edit and delete buttons would be.

> **CHECKPOINT** Log in as a student and confirm the Teachers page looks like the figure. Then get a 403 on purpose: Chapter 10, step 6, does it in one command. Seeing the server refuse something the interface already hid is the moment the two-layer arrangement makes sense.

What you have now

You know how a header becomes `request.user`, and that the role is looked up rather than carried. You know what failing closed means and which single setting does it. You can read all four permission classes, explain why students may create submissions but not edit them, tell 401 from 403 by what each means rather than what it is called, and say precisely why the same rules exist in two languages in this codebase.

Next: the two ways a password changes.

CHAPTER 9

Changing and Resetting a Password

Two situations that look similar and are completely different problems. Getting them mixed up is how password reset flows end up being the weakest part of an otherwise careful app.

Two different problems

	CHANGE	RESET
The situation	You know your password and want a new one	You have forgotten it and cannot get in
Are you signed in?	Yes	No — that is the whole problem
How you prove it is you	Type the current password	Read an email only you can read
Endpoint	`/api/change-password/`	`/api/password-reset/` then `/api/password-reset-confirm/`
Permission	`IsAuthenticated`	`AllowAny`

The second row is the interesting one. A reset endpoint *must* be open to anonymous callers, because a locked-out person has no token. That makes it the softest surface on the whole API, and everything odd about its design is a response to that fact.

Changing it when you know it

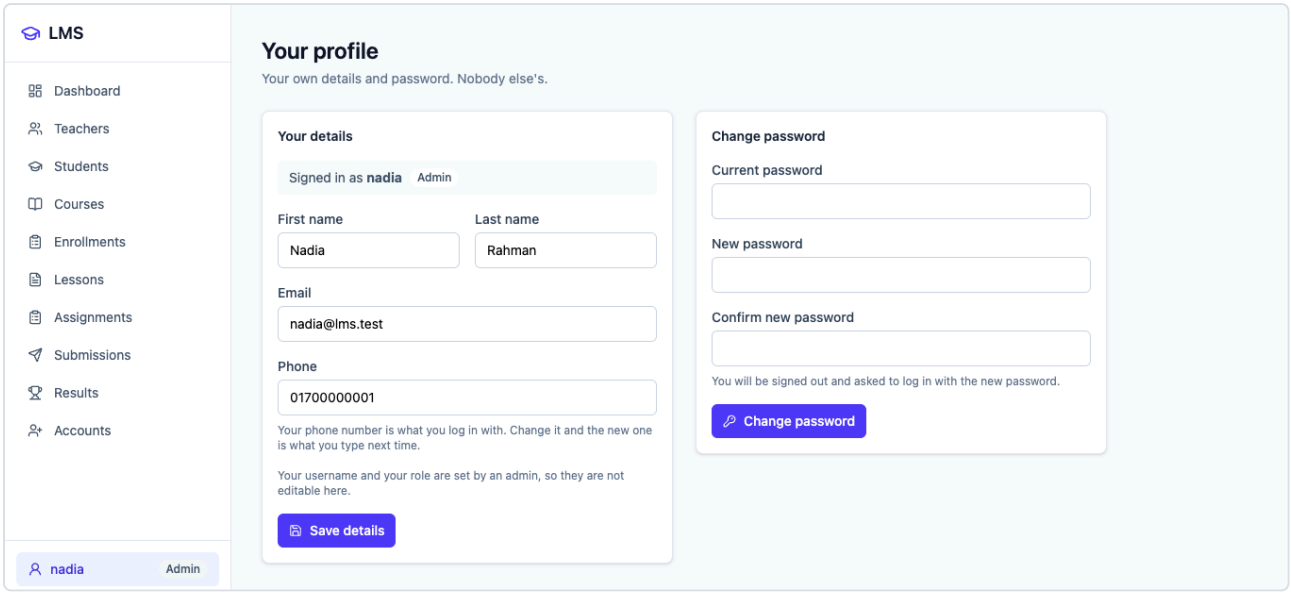

Figure 9.1 The profile page. Two forms, two jobs. The right-hand one asks for the current password first.

backend/backend/serializers.py

```
class ChangePasswordSerializer(serializers.Serializer):
    current_password = serializers.CharField(write_only=True)
    new_password = serializers.CharField(write_only=True)
    confirm_password = serializers.CharField(write_only=True)

    def validate_current_password(self, value):
        # Knowing the old password is what makes this yours to change. Without
        # it, a borrowed browser tab would be enough to take the account.
        if not self.context["request"].user.check_password(value):
            raise serializers.ValidationError("That is not your current password.")
        return value

    def validate(self, attrs):
        if attrs["new_password"] != attrs["confirm_password"]:
            raise serializers.ValidationError({
                "confirm_password": "The two new passwords do not match."
            })

        if attrs["new_password"] == attrs["current_password"]:
            raise serializers.ValidationError({
                "new_password": "The new password has to be different from the old one."
            })

        validate_password(attrs["new_password"], user=self.context["request"].user)
        return attrs
```

Note the two kinds of validation method. `validate_current_password` checks one field on its own. Plain `validate` runs afterwards and can see *all* the fields at once, which is the only way to compare two of them. Any rule involving more than one field goes there.

The comment on the first one is the reason this endpoint exists in the form it does. A live session is not enough to change a password. If it were, walking away from an unlocked laptop would mean losing the account permanently rather than temporarily — the person at your desk could lock you out of your own login. Asking for the current password means the attacker needs the secret, not just the chair.

`self.context["request"]` is how a serializer gets at the caller. It is not automatic; the view passes it in:

backend/backend/views.py

```
class ChangePasswordView(APIView):
    """Change your own password, knowing the old one."""
    permission_classes = [IsAuthenticated]

    def post(self, request):
        serializer = ChangePasswordSerializer(
            data=request.data,
            context={'request': request},
        )
        serializer.is_valid(raise_exception=True)

        user = request.user
        user.set_password(serializer.validated_data['new_password'])
        user.save()

        # Refresh tokens issued earlier are retired. The access token already
        # in the caller's hands cannot be revoked — JWTs are not looked up on
        # each request — so the frontend signs you out and makes you log in
        # again with the new password.
        blacklist_user_tokens(user)

        return Response(
            {'detail': 'Password changed. Please log in again.'},
            status=status.HTTP_200_OK,
        )
```

Two safety properties fall out of five lines. The view operates on `request.user` and nothing else, so there is no field anywhere that could aim it at another account. And `set_password` is followed immediately by `save()` — Chapter 4's trap, avoided.

The blacklist, and what it can and cannot retire

backend/backend/views.py

```
def blacklist_user_tokens(user):
    try:
        from rest_framework_simplejwt.token_blacklist.models import (
            OutstandingToken,
            BlacklistedToken,
        )

        tokens = OutstandingToken.objects.filter(user=user)

        for token in tokens:
            BlacklistedToken.objects.get_or_create(token=token)

    except Exception:
        pass
```

The `token_blacklist` app — switched on in `INSTALLED_APPS` back in Chapter 2 — keeps two tables. `OutstandingToken` records every **refresh** token that has been issued. `BlacklistedToken` lists the ones that must no longer work.

Here is the limit, and it is Chapter 6's cost showing up for the last time: **only refresh tokens are recorded, so only refresh tokens can be blacklisted.** Access tokens are never looked up on a request — the whole speed argument for JWTs is that a signature check needs no database — so there is nothing to check a blacklist against. The access token already in someone's hands keeps working until `exp`.

Which means, for this app specifically: since it never uses refresh tokens at all, this function is close to ceremonial. It is correct, it costs nothing, and it would start mattering the day a refresh endpoint is added. Leaving it in is a reasonable bet on the future.

The bare `except Exception: pass` is worth a word too. It is there so that a project without the blacklist app installed still changes passwords rather than crashing. That is a deliberate trade — swallowing errors hides problems, and in security code it is usually the wrong instinct. Here the swallowed failure cannot cause harm, because the password has already been changed by the time it runs.

The deliberate sign-out

Since the current access token survives a password change, the frontend does the only honest thing available: it throws the session away itself.

frontend/src/pages/Profile.jsx

```
      // The old token stays technically valid until it expires, so the honest
      // thing is to end the session here and make them sign in with the new
      // password.
      logOut();
      navigate("/login", { replace: true });
```

Which leaves one rough edge, and this app has it: you land on the login screen with no explanation, and a login page that appears out of nowhere reads like something failed, when in fact the change worked.

Carrying a message across that sign-out is harder than it looks, and the reason is worth knowing. `logOut()` flips `isLoggedIn` to false, which re-renders the Profile page inside `ProtectedRoute`, which fires *its own* redirect to `/login` and replaces the history entry. That redirect happens first and carries only the `from` path. Any state attached to the `navigate()` call on the line below never arrives.

So the message has to be parked somewhere no redirect can reach. `sessionStorage` is the usual shelf — localStorage's short-lived cousin, same interface, cleared when the tab closes — with the login page reading it on its first render and deleting it as it reads, so the notice appears exactly once and does not come back on the next visit. An earlier version of this project did precisely that, in a small `src/flash.js`; it was removed along with the rest of that file when the app moved to banners the reader closes by hand. Putting it back is a contained exercise, and now you know which three pieces it needs.

Resetting it when you have forgotten it

Now the harder problem. The person cannot prove anything: no password, no token. The only thing they still control is their email inbox, so the flow is built on that.

Step one — ask for a link:

backend/backend/views.py

```
class PasswordResetRequestView(APIView):
    permission_classes = [AllowAny]

    def post(self, request):
        serializer = PasswordResetRequestSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        email = serializer.validated_data["email"]
        user = User.objects.filter(email=email, is_active=True).first()

        if user:
            uid = urlsafe_base64_encode(force_bytes(user.pk))
            token = default_token_generator.make_token(user)

            reset_link = (
                f"{settings.FRONTEND_PASSWORD_RESET_URL}"
                f"?uid={uid}&token={token}"
            )

            send_mail(
                subject="Reset your password",
                message=f"Click the link below to reset your password:\n\n{reset_link}",
                from_email=settings.DEFAULT_FROM_EMAIL,
                recipient_list=[user.email],
                fail_silently=False,
            )

        return Response(
            {
                "detail": "If an account exists with this email, a password reset link has
been sent."
            },
            status=status.HTTP_200_OK,
        )
```

Step two — use the link:

backend/backend/views.py

```
class PasswordResetConfirmView(APIView):
    permission_classes = [AllowAny]

    def post(self, request):
        serializer = PasswordResetConfirmSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        user = serializer.validated_data["user"]
        new_password = serializer.validated_data["new_password"]

        user.set_password(new_password)
        user.save()

        blacklist_user_tokens(user)

        return Response(
            {
                "detail": "Password has been reset successfully. Please login again."
            },
            status=status.HTTP_200_OK,
        )
```

And the checking, all of which happens in the serializer:

backend/backend/serializers.py

```python
class PasswordResetConfirmSerializer(serializers.Serializer):
    uid = serializers.CharField()
    token = serializers.CharField()
    new_password = serializers.CharField(write_only=True)
    confirm_password = serializers.CharField(write_only=True)

    def validate(self, attrs):
        if attrs["new_password"] != attrs["confirm_password"]:
            raise serializers.ValidationError({
                "confirm_password": "Passwords do not match."
            })

        try:
            uid = force_str(urlsafe_base64_decode(attrs["uid"]))
            user = User.objects.get(pk=uid, is_active=True)
        except Exception:
            raise serializers.ValidationError({
                "uid": "Invalid reset link."
            })

        if not default_token_generator.check_token(user, attrs["token"]):
            raise serializers.ValidationError({
                "token": "Invalid or expired reset token."
            })

        validate_password(attrs["new_password"], user=user)

        attrs["user"] = user
        return attrs
```

The last line before the return is a small trick worth noticing: the serializer puts the `user` object it found into `attrs`, so the view can pull it straight out of `validated_data` instead of looking it up a second time.

uid and token, decoded

The link carries two values and they do different jobs.

uid answers "which account?". It is the user's primary key, base64url-encoded so it survives being in a URL. User 7 becomes `Nw`. There is nothing secret about it — encoding, again, is not encryption — and anybody can decode it in one line:

```
from django.utils.http import urlsafe_base64_decode
from django.utils.encoding import force_str

force_str(urlsafe_base64_decode("Nw"))     # '7'
```

token is the part that proves anything, and it is not a JWT. It comes from Django's `default_token_generator`, which builds a hash out of four ingredients:

- the user's primary key,
- the current password hash,
- the account's `last_login` timestamp,
- the time the token was made,

all signed with `SECRET_KEY`. That combination gives the token three properties for free, without a single row being stored anywhere:

PROPERTY	WHY IT HOLDS
Single use	The password hash is an ingredient. Once the password changes, the hash changes, and the token no longer verifies.
Expires	The creation time is an ingredient, and `check_token` compares it against `PASSWORD_RESET_TIMEOUT`.
Unforgeable	Signed with `SECRET_KEY`, which the requester does not have.

The single-use property is the elegant bit. Nobody wrote code to mark the token as used; it invalidates itself as a side effect of the thing it was for. And this project sets the window tight:

backend/lms/settings.py

```
FRONTEND_PASSWORD_RESET_URL = os.getenv("FRONTEND_PASSWORD_RESET_URL",
"http://localhost:5173/reset-password")

PASSWORD_RESET_TIMEOUT = 60 * 60  # 1 hour
```

Django's own default is three days. One hour is a deliberate tightening, and the trade is the usual one: a link that reaches somebody's phone while they are asleep will have expired by morning.

WATCH THE PORT

`FRONTEND_PASSWORD_RESET_URL` defaults to port **5173**, but this project's Vite dev server runs on **5180** (see `frontend/vite.config.js`). The emailed link will point at a port with nothing behind it unless you set the environment variable. There is also no `/reset-password` page in `App.jsx` yet — the backend half of this flow is complete and the frontend half is not, which is worth knowing before you go looking for a screen that does not exist.

The answer that tells you nothing

Look again at the shape of the request view. The email lookup and the sending are inside `if user:`. The response is outside it. Both paths return the same sentence and the same 200:

```
{"detail": "If an account exists with this email, a password reset link has been sent."}
```

This is the same reasoning as the vague login error in Chapter 4, applied to a different endpoint. If the answer differed, anyone could type addresses into the form and learn which ones have accounts here. The phrasing is careful, too — "if an account exists" is not a lie in either case, which matters, because a message that says "sent!" when nothing was sent is a message users learn to distrust.

`is_active=True` appears in both halves of the flow, so a disabled account cannot be reactivated through the back door of a password reset.

Where the email goes in development

backend/lms/settings.py

```
EMAIL_BACKEND = os.getenv("EMAIL_BACKEND",
"django.core.mail.backends.console.EmailBackend")
DEFAULT_FROM_EMAIL = os.getenv("DEFAULT_FROM_EMAIL", "no-reply@example.com")
EMAIL_HOST = os.getenv("EMAIL_HOST", "smtp.gmail.com")
EMAIL_PORT = int(os.getenv("EMAIL_PORT", "587"))
EMAIL_HOST_USER = os.getenv("EMAIL_HOST_USER", "asif1920001@gmail.com")
EMAIL_HOST_PASSWORD = os.getenv("EMAIL_HOST_PASSWORD", "")
EMAIL_USE_TLS = os.getenv("EMAIL_USE_TLS", "True") == "True"
```

The default backend is `console.EmailBackend`, which does not send anything. It prints the whole email into the terminal where `runserver` is running. For development that is ideal: no mail account to configure, and the reset link is right there to copy.

So the flow you can actually try today is: POST an email address to `/api/password-reset/`, look in the Django terminal, copy `uid` and `token` out of the printed link, and POST them to `/api/password-reset-confirm/` along with a new password.

> **CHECKPOINT** Do exactly that, then try the *same* uid and token a second time. The answer is `{"token": ["Invalid or expired reset token."]}`, because the password hash that helped build the token has changed. That is the single-use property, demonstrated in two commands.

What you have now

You know why changing a password asks for the old one, why the app signs you out afterwards, and why the login page you land on has nothing to say about it. You know what the blacklist can and cannot retire and why. You can explain both halves of the reset flow, what `uid` and `token` each contribute, why the token is single-use without anything being stored, and why the endpoint refuses to tell you whether the address was real.

Next: doing all of it yourself, from a terminal.

CHAPTER 10

Do It Yourself: The Whole Flow

Reading about a token is one thing. Holding one in your terminal, watching a request fail without it and succeed with it, is what makes the subject stop being abstract. Type this chapter; do not skim it.

What curl is

`curl` is a program that makes a web request and prints the answer. It comes with macOS and Linux; on Windows it is in PowerShell and in Git Bash. It matters here because a browser *cannot* do what you are about to do — typing an API address into the address bar sends a request with no `Authorization` header, so you would only ever see 401. curl lets you attach one.

Three options do everything in this chapter:

OPTION	MEANING
`-X POST`	Use the POST method rather than GET
`-H "..."`	Add a header. Repeatable.
`-d '{...}'`	Send this text as the body
`-i`	Print the status line and headers as well as the body

0. Start the server

```
cd backend
source .venv/bin/activate          # Windows: .venv\Scripts\activate
python manage.py runserver 8001
```

Leave it running and open a second terminal for everything below. The port is 8001 because that is the one `api.js` calls; if you prefer another, run Django there and set `VITE_API_URL` to match (Chapter 7 has the details).

You will need an admin account with a Profile. If you have not made one, Chapter 2 shows how. The examples below use the phone number `01711110001` and the password `admin-pass-1`; substitute your own throughout.

1. Knock without a token

```
curl -i http://127.0.0.1:8001/api/teacher/
```

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer realm="api"
Content-Type: application/json

{"detail":"Authentication credentials were not provided."}
```

There is Chapter 8's 401: not "you may not", but "I do not know who you are". Note the `WWW-Authenticate` header telling you what kind of credential is expected. Reading a list is open to any signed-in account — and you are not one yet.

2. Log in as the admin

```
curl -X POST http://127.0.0.1:8001/api/login/ \
  -H "Content-Type: application/json" \
  -d '{"phone": "01711110001", "password": "admin-pass-1"}'
```

```
{
  "message": "Login successful",
  "user_id": 1,
  "username": "nadia",
  "role": "admin",
  "tokens": {
    "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
}
```

Copy the **access** string — the one under `tokens`, not the refresh — and put it in a shell variable so you do not have to paste it every time:

```
TOKEN='eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...paste the whole thing...'
```

Single quotes, no spaces around the `=`, and the entire string with nothing trimmed off the end. A truncated token fails the signature check and gives you a 401 that looks exactly like not being logged in.

TIP

If you have `jq` installed, the whole step is one line:

```
TOKEN=$(curl -s -X POST http://127.0.0.1:8001/api/login/ \
  -H "Content-Type: application/json" \
  -d '{"phone": "01711110001", "password": "admin-pass-1"}' | jq -r
.tokens.access)
```

Note the path `.tokens.access`. Chapter 5's nesting, one more time.

3. Use the token

The same request as step 1, with one header added:

```
curl -i http://127.0.0.1:8001/api/teacher/ \
  -H "Authorization: Bearer $TOKEN"
```

```
HTTP/1.1 200 OK
Content-Type: application/json

[{"id":1,"name":"Rina
Haque","email":"rina@example.com","subject":"Physics","is_active":true}]
```

That one header is the difference between a stranger and user 1. Everything in this book exists to make that line possible and trustworthy.

Now ask the API who it thinks you are:

```
curl http://127.0.0.1:8001/api/profile/ -H "Authorization: Bearer $TOKEN"
```

```
{
  "message": "successfully fetched this user",
  "user": {
    "user_id": 1,
    "username": "nadia",
    "email": "nadia@example.com",
    "first_name": "Nadia",
    "last_name": "Islam",
    "phone": "01711110001",
    "role": "admin"
  }
}
```

The server read `user_id` out of the token's payload, loaded that row, and built this answer. No session, no cookie, no memory of step 2.

4. Create an account

```
curl -i -X POST http://127.0.0.1:8001/api/register/ \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
        "username": "kabir",
        "email": "kabir@example.com",
        "password": "correct-horse-9",
        "phone": "01711110009",
        "role": "student",
        "first_name": "Kabir",
        "last_name": "Ahmed"
      }'
```

```
HTTP/1.1 201 Created

{"id":9,"username":"kabir","email":"kabir@example.com",
 "first_name":"Kabir","last_name":"Ahmed","role":"student"}
```

`201 Created` rather than 200, because something new exists. No password and no phone in the answer — both `write_only`, as Chapter 3 promised. Two rows were written inside one transaction.

Try the same command a second time and watch the serializer refuse it:

```
{"username":["That username is already taken."],
 "phone":["This phone number is already registered."]}
```

Both problems in one answer, each under its field name. That is why the frontend's `readError` walks the whole object instead of reading one message.

5. Log in as the new person

```
curl -X POST http://127.0.0.1:8001/api/login/ \
  -H "Content-Type: application/json" \
  -d '{"phone": "01711110009", "password": "correct-horse-9"}'
```

Note what you did *not* type: `kabir`. The username never appears at login. Keep this token separately:

```
STUDENT_TOKEN='...the access token from that answer...'
```

And confirm the role came back as `student`.

6. Get yourself a 403

Reading first, which should work:

```
curl -i http://127.0.0.1:8001/api/teacher/ \
  -H "Authorization: Bearer $STUDENT_TOKEN"
```

`200 OK`. Reading is open to any signed-in account. Now writing:

```
curl -i -X POST http://127.0.0.1:8001/api/teacher/ \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $STUDENT_TOKEN" \
  -d '{"name": "Sneaky", "email": "s@example.com", "subject": "Nothing", "is_active":
true}'
```

```
HTTP/1.1 403 Forbidden

{"detail":"Only an admin can change teacher and student records."}
```

That sentence is the `message` attribute of `AdminWrites`, straight out of `permissions.py`. And now try creating an account as a student:

```
curl -i -X POST http://127.0.0.1:8001/api/register/ \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $STUDENT_TOKEN" \
  -d '{"username": "x", "email": "x@example.com", "password": "correct-horse-9", "phone":
"0171111000X"}'
```

```
HTTP/1.1 403 Forbidden

{"detail":"Only an admin can create accounts."}
```

This is the important one. The React app never draws that button for a student, and the sidebar has no Accounts link — but you just went around the entire frontend, straight at the API, and the server refused anyway. That is the difference between hiding a door and locking it.

7. Break the token on purpose

Change one character in the middle of the token — anywhere in the payload or the signature — and try again:

```
curl -i http://127.0.0.1:8001/api/teacher/ \
  -H "Authorization: Bearer ${TOKEN}x"
```


```
{"detail":"Given token not valid for any token type",
 "code":"token_not_valid",
 "messages":[{"token_class":"AccessToken","token_type":"access","message":"Token is
invalid"}]}
```


401, not 403. The signature no longer matches the contents, so the server does not know who you are at all. This is Chapter 6's integrity guarantee, tested by hand: you cannot edit a claim, because you cannot re-sign it.

Two more variations worth a minute each:

```
# no Bearer prefix
curl -i http://127.0.0.1:8001/api/teacher/ -H "Authorization: $TOKEN"

# the wrong prefix, common in other frameworks
curl -i http://127.0.0.1:8001/api/teacher/ -H "Authorization: Token $TOKEN"
```

Both give 401 with "Authentication credentials were not provided" — the header was there, but not in the form `AUTH_HEADER_TYPES` accepts, so it was ignored entirely. If you ever see that message while staring at a request that clearly has a token in it, this is why.

8. Change a password

```
curl -i -X POST http://127.0.0.1:8001/api/change-password/ \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $STUDENT_TOKEN" \
  -d '{"current_password": "correct-horse-9",
       "new_password": "battery-staple-7",
       "confirm_password": "battery-staple-7"}'
```


```
{"detail": "Password changed. Please log in again."}
```


Now three experiments, in order:

1. **Use the old token again** — `curl http://127.0.0.1:8001/api/teacher/ -H "Authorization: Bearer $STUDENT_TOKEN"`. It still works. The access token was never cancelled; the browser app only *pretends* to log you out by deleting its copy. Chapter 6's cost, in your own terminal.
2. **Log in with the old password** — refused. The stored hash has changed.
3. **Get the validators to bite** — try changing to `12345678`:

   ```
   {"new_password":["This password is entirely numeric.","This password is too common."]}
   ```

   Two of Chapter 4's four validators, both firing on one value.

> **CHECKPOINT** You have gone through the entire subject of this book with nothing but a terminal: refused without a token, logged in, used a token, created an account, been refused by role, broken a signature, and changed a password. If step 6 and step 8's first experiment both made sense, you understand this system better than most people who have shipped one.

What you have now

You can drive the whole API by hand, which means you can now tell a frontend bug from a backend bug in about thirty seconds — run the same call in curl and see which half is wrong. You have seen 401 and 403 arrive for the reasons Chapter 8 said they would, and you have watched a token survive a logout.

Next: what to do when something goes wrong that you did not cause on purpose.

CHAPTER 11

When It Goes Wrong

Every failure in this area produces one of about a dozen messages. This chapter lists them with what each actually means. Skim it once now so the words look familiar, and come back to it when something breaks.

Read the status code first

Before reading any message, look at the number. It narrows the problem to a quarter of the codebase.

CODE	MEANS	LOOK AT
400	Your request was understood and rejected	The body you sent: missing fields, bad values, wrong password
401	I do not know who you are	The `Authorization` header, and whether the token is intact and unexpired
403	I know who you are, and no	The role on your Profile, and the permission class on that endpoint
404	No such address, or no such row	The URL, especially the trailing slash
405	Right address, wrong method	POST versus PUT versus PATCH
500	The server crashed	The traceback in the `runserver` terminal. This one is a bug, not a refusal.
No response at all	Nothing was reached	Is Django running? Right port? CORS?

In the browser, find the number in developer tools → Network, then click the failing request. With curl, add `-i`.

Errors at login

MESSAGE	CODE	WHAT IS ACTUALLY WRONG
`Invalid phone or password`	400	One of three things: no Profile has that phone number, the password is wrong, or the account has a User row but no Profile (a fresh `createsuperuser`). The message is vague on purpose — Chapter 4. Check in the Django admin which case it is.
`{"phone": ["This field is required."]}`	400	The field was missing or misspelled in the body. Note it is `phone`, not `username` or `email`.
`Could not reach the server. Is Django running on http://127.0.0.1:8001?`	—	The frontend's own message when `fetch` never got an answer. Django is not running, or is on a different port from the one `api.js` is calling. See the port note below.
`MultipleObjectsReturned` in the terminal	500	Two Profiles share a phone number, which the unique constraint should prevent. It means rows were made outside the app, probably in a shell.

THE PORT, ONCE MORE

`api.js` defaults to `http://127.0.0.1:8001/api`, and the README starts Django with `python manage.py runserver 8001` to match. Plain `runserver` listens on **8000**, and then nothing works. Fix it in one of two places — start Django on 8001, or put a `frontend/.env` file containing `VITE_API_URL=http://127.0.0.1:8000/api` next to it — and restart the Vite dev server afterwards, because environment variables are read at startup.

Errors at registration

MESSAGE	CODE	WHAT IS ACTUALLY WRONG
`Only an admin can create accounts.`	403	You are signed in as a teacher or student. `IsAdmin` refused you.
`Authentication credentials were not provided.`	401	No token on the request at all. Registration is not public here.
`This phone number is already registered.`	400	Another Profile has it. Phone numbers are unique because logging in depends on it.
`That username is already taken.`	400	Django's `username` column is unique.
`This password is too short. It must contain at least 8 characters.`	400	MinimumLengthValidator.
`This password is too common.`	400	CommonPasswordValidator — it is in Django's list of 20,000.
`The password is too similar to the username.`	400	UserAttributeSimilarityValidator.
`{"email": ["Enter a valid email address."]}`	400	`EmailField` checking the format. It does not check that the address exists.
`{"role": ["\"boss\" is not a valid choice."]}`	400	`ChoiceField`. Only `admin`, `teacher`, `student`.

An account was created but the person cannot log in? Two likely causes, both from Chapter 3: the Profile row is missing (something bypassed the register endpoint), or the password was stored unhashed by `User.objects.create` instead of `create_user`. Check the `password` column — if it does not start with `pbkdf2_sha256$`, that is your answer.

Errors on every other call

MESSAGE	CODE	WHAT IS ACTUALLY WRONG
`Authentication credentials were not provided.`	401	No header, or a header the server ignored: missing `Bearer`, wrong prefix, missing space. Chapter 10, step 7.
`Given token not valid for any token type`	401	The token arrived but failed verification: truncated when pasted, altered, or signed with a different `SECRET_KEY`.
`Token is expired`	401	`exp` has passed. Eight hours in this project. Log in again.
`Your session has expired. Please log in again.`	—	The frontend's wording for any 401, thrown by `api.js` after it clears the session.
`Your role does not allow this change.`	403	The base `RoleWritePermission` message, from an endpoint that did not override it.
`Only an admin can change teacher and student records.`	403	`AdminWrites`. You are a teacher or a student.
`Only a teacher or an admin can change course material and grades.`	403	`TeachingStaffWrites`. You are a student.
`Students can hand work in, but cannot change or remove a submission.`	403	`SubmissionWrites`. A student tried PUT or DELETE rather than POST.
`Not found.`	404	Usually a missing trailing slash. Django's URLs end in `/`, and `/api/teacher` is not `/api/teacher/`.

Errors in the browser console

These never reach your Python code, which is what makes them confusing the first time.

MESSAGE	WHAT IT MEANS
`Access to fetch at 'http://127.0.0.1:8001/api/login/' from origin 'http://localhost:5180' has been blocked by CORS policy`	The browser's same-origin rule. The frontend's address is not in `CORS_ALLOWED_ORIGINS`, or `corsheaders` is not installed, or its middleware is in the wrong place. It must sit above `CommonMiddleware`.
`Failed to fetch`	The request did not reach a server. Django is down, or the port is wrong. The same condition `api.js` catches and rewords.
`Cannot read properties of undefined (reading 'access')`	Something read `data.access` instead of `data.tokens.access`. Chapter 5.
`useAuth must be used inside an AuthProvider`	A component called `useAuth()` from outside the provider. In this app everything is inside it, via `main.jsx`.
Bounced to `/login` immediately after logging in	`saveSession` did not run, or localStorage is unavailable — some browsers block it in private mode.
Signed in, but the Accounts link is missing and you are an admin	The saved `lms_user` has no `role`. Reload once and let the `/profile/` top-up in `AuthContext.jsx` fill it in, or log out and back in.

The five-minute checklist

Something is wrong and you do not know where. In order:

1. **Is Django running, and on the port the frontend is calling?** Look at the terminal, then at `BASE_URL` in `api.js`.
2. **What is the status code?** Network tab, or `-i` in curl. The table at the top of this chapter turns it into a place to look.
3. **Does the same call work in curl?** If yes, the bug is in the frontend. If no, it is in the backend. This single test saves more time than any other.
4. **Is there a token in localStorage, and is it whole?** Developer tools → Application. Decode the middle section (Chapter 6) and check `exp`.
5. **Does the account have a Profile, with the role you expect?** `http://127.0.0.1:8001/admin/backend/profile/`. A missing Profile explains a surprising number of symptoms.
6. **Anything in the `runserver` terminal?** Every request is logged with its status code, and any 500 prints a full traceback there.

> **THE ONE-LINE DIAGNOSIS**
>
> Step 3 is the one to internalise. "Works in curl, fails in the browser" means a header, CORS or a URL problem. "Fails in both" means the server is doing exactly what you told it to, and the answer is in `permissions.py` or a serializer.

What you have now

You can turn any error this part of the app produces into a specific file to open. You know which messages come from Django, which from DRF, which from the frontend's own code, and which from the browser before either one was reached.

One chapter left: the honest list of what this system does not do.

CHAPTER 12

What This Project Does Not Do

A guide that only explains what a system does teaches you to trust it too much. This chapter is the other half: what is deliberately missing, what is a genuine risk, and what would have to change before this app faced the internet. Knowing the gaps in a system you have read is the fastest way to learn what a complete one contains.

A teaching app, not a bank

Everything below is fine for a project running on your laptop. None of it is fine for a server holding other people's data. The line between those two situations is not a matter of scale; it is the moment somebody who is not you can reach it.

The settings that must change

Three lines, and they are the most serious items on the list.

backend/lms/settings.py

```
SECRET_KEY = 'django-insecure-@hg+%+(*w2m0jh=8$ksnpsoe+dv2jb+py3@1b(3vhohjb7cx8!'

DEBUG = True

EMAIL_HOST_USER = os.getenv("EMAIL_HOST_USER", "asif1920001@gmail.com")
```

LINE	WHY IT MATTERS	WHAT TO DO
`SECRET_KEY` in the file	Every token is signed with it. Anybody who reads the repository can sign a token claiming to be user 1, and the server will believe it. This is complete authentication bypass, not a warning to tidy up later.	Read it from an environment variable, generate a new one for the real deployment, and never commit either.
`DEBUG = True`	Any crash returns a full traceback to the caller: file paths, local variables, and settings. It is a guided tour of your server.	`DEBUG = os.getenv("DEBUG", "False") == "True"`, and set `ALLOWED_HOSTS` properly.
A real email address as a default	Harmless on its own, but it is a personal address baked into a public file, and it sits next to `EMAIL_HOST_PASSWORD`, which one day somebody will fill in.	No defaults for anything about mail. Let it fail loudly if unconfigured.

The pattern to take away: **secrets belong in the environment, never in the repository.** The rest of the settings file already reads from `os.getenv`; these three are the stragglers.

No rate limit on login

`/api/login/` will answer as fast as you can ask. Nothing counts failed attempts, nothing slows down after ten, nothing locks an account, and there is no CAPTCHA. A script can try thousands of passwords against one phone number and the only thing standing in its way is the million-iteration hash from Chapter 4, which slows each guess but does not stop the flood.

This is the most likely way this app would actually be broken into, and it is also the cheapest to fix. DRF has throttling built in:

```
REST_FRAMEWORK = {
    ...
    "DEFAULT_THROTTLE_CLASSES": ("rest_framework.throttling.AnonRateThrottle",),
    "DEFAULT_THROTTLE_RATES": {"anon": "20/hour"},
}
```

The same gap applies to `/api/password-reset/`, where an unthrottled endpoint also means anyone can make your server send mail to any address as often as they like.

The token in localStorage

Chapter 7 put the access token in localStorage, which is the common choice and is not free of risk. Any JavaScript running on the page can read it — including anything injected through a **cross-site scripting** hole, where attacker-supplied text ends up executed as code. A stolen token is a working login for up to eight hours.

The alternative is an `HttpOnly` cookie, which JavaScript cannot read at all. That closes this hole and opens another, **cross-site request forgery**, which then needs its own defence. Neither choice is free; the difference is which attack you are prepared for.

React escapes text by default, which is most of an XSS defence, so this app is not obviously vulnerable. The point is that "not obviously vulnerable" is the entire protection standing between an injected script and every account's token.

No refresh endpoint

The consequence chain, one last time, because it is the clearest example in this codebase of one decision setting up the next:

```
no refresh endpoint
      |
      v
a 5-minute access token would be unusable
      |
      v
access token widened to 8 hours
      |
      v
a stolen token is useful for 8 hours
      |
      v
logout cannot cancel it, so logout is
"delete our copy"
      |
      v
changing a password cannot cancel it either,
so the app signs you out and asks you to
log in again
```

Adding `TokenRefreshView` from Simple JWT is about ten lines and would let the access token drop to five minutes. It also needs the frontend to catch a 401, call refresh, and retry the original request — which is the real work, and the reason it was skipped.

No email verification, no two-factor

Several things a production account system has, which this one does not:

- **Email verification.** Nothing checks that `rina@example.com` is reachable, or Rina's. Since the reset flow trusts that address completely, a typo at registration means the account can be recovered by whoever owns the address that was typed.
- **Two-factor authentication.** One password is the whole of the security on every account, including admins.
- **An audit trail.** Nothing records who created an account, who changed a role, or when. For a system where roles decide who can alter grades, that is a real gap.
- **Account lockout or notification.** Nobody is told when their password changes, and nothing reacts to a hundred failed logins.
- **A logout endpoint.** Discussed to death; there is nothing useful for it to do without refresh tokens in play.

The submission hole

The most interesting gap, because it comes from the data model rather than from a missing setting. Chapter 8 quoted the docstring:

> A student cannot edit or delete a submission, because nothing links a Student row to a login account — so the server has no way to tell whose work it is.

Look at the two tables side by side and the problem is plain:

```
  auth_user + backend_profile      backend_student
  ---------------------------      ---------------
  who can log in                   who is in the school
  username, password, phone,       name, email, roll_number,
  role                             enrollment_date

              (no link between them)
```

A logged-in student is a `User`. A submission belongs to a `Student`. The server cannot tell whether the person POSTing a submission is the student named in it, so it allows the create and forbids everything else. That is the largest amount of trust the current schema can support — and it still means one student can hand in work under another student's name.

The fix is one field:

```python
class Student(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE, null=True, blank=True)
    ...
```

With that link, the submission endpoint could set the student from `request.user` rather than believing the request body, and object-level permissions could let people edit their own work and nobody else's. It would also let the app show a student *their* grades instead of everyone's, which is the feature this gap is really blocking.

This is worth sitting with, because it is the general shape of a whole class of security bugs: **the permission check can only be as precise as the data model allows.** No amount of cleverness in `permissions.py` can answer "is this your submission?" while the answer is not recorded anywhere.

What you have now

You have the honest list: three settings that must move to the environment, no rate limiting anywhere, a token stored where scripts can read it, no refresh flow and the chain of compromises that follows from it, no email verification or second factor, and one missing foreign key that caps how precise the permission rules can ever be.

Read that list as a curriculum rather than a complaint. Each item is a real system's real feature, and now you know exactly which problem each one exists to solve.

APPENDIX A

Glossary

Every term this book defines, in one place, with the chapter that explains it properly.

TERM	MEANING	CH.
Access token	The short-lived token sent with every request. Eight hours here.	6
Account enumeration	Learning which phone numbers or emails have accounts, by noticing that the app answers differently for real ones. Prevented by identical messages.	4, 9
AllowAny	DRF permission class meaning "no sign-in required". On login and the two reset endpoints.	5, 8
AnonymousUser	What `request.user` is when nobody is signed in. Its `is_authenticated` is `False`.	8
Authentication (authn)	Working out who somebody is. Login.	1
Authorisation (authz)	Deciding what that person may do. Roles and permissions.	1, 8
base64url	Writing arbitrary text using only URL-safe characters. Encoding, not encryption — anyone can reverse it.	6
Bearer	The word before the token in the `Authorization` header. It means "whoever holds this is treated as its subject".	6, 7
Blacklist	Simple JWT's table of refresh tokens that must no longer work. Access tokens cannot be listed there.	9
Claim	One name-and-value pair inside a token's payload, such as `exp` or `user_id`.	6
Context (React)	A value provided high in the component tree and readable by any component below, without passing it down by hand.	7
CORS	Cross-Origin Resource Sharing. The server's list of other addresses whose pages may call it. Needed because the frontend and backend are on different ports.	1, 11
CSRF	Cross-site request forgery. An attack that cookie-based auth must defend against and token-based auth largely sidesteps.	12
curl	A command-line program for making web requests. The way to test an API without a browser.	10
exp	The expiry claim in a JWT, as seconds since 1 January 1970.	6
Fail closed	When unsure, refuse. Why `DEFAULT_PERMISSION_CLASSES` is `IsAuthenticated` and why `role_of` returns student for an unknown account.	8
Hash	A one-way calculation. Same input, same output; no way back from the output.	4
HttpOnly cookie	A cookie JavaScript cannot read. The main alternative to keeping a token in localStorage.	12
iat	"Issued at". The claim recording when a token was signed.	6

TERM	MEANING	CH.
IsAuthenticated	DRF permission class meaning "any signed-in account". The project-wide default.	8
JSON	The text format APIs speak: braces, quoted names, values.	3
JWT	JSON Web Token. A signed, readable note in three base64url parts.	6
jti	A unique id for one token. What the blacklist tracks.	6
localStorage	Per-site text storage in the browser that survives reloads and restarts. Where the access token lives.	7
Middleware	Code that runs around every Django request. `corsheaders` is one, and its order in the list matters.	11
OneToOneField	A link where each row on either side matches at most one row on the other. Joins `Profile` to `User`.	2
PBKDF2	The deliberately slow password hashing algorithm Django uses by default. A million rounds in Django 5.2.	4
Permission class	An object with `has_permission()` that DRF calls before a view runs.	8
Profile	This project's own table holding the phone number and role, one row per `User`.	2
Refresh token	A longer-lived token whose only job is to mint new access tokens. Issued here, never used.	6
request.user	The account DRF worked out from the token, available in every view.	8
Role	`admin`, `teacher` or `student`. Stored on the Profile; never inside the token.	2, 8
SAFE_METHODS	DRF's tuple `("GET", "HEAD", "OPTIONS")` — the methods that only read.	8
Salt	Random text mixed into a password before hashing, so identical passwords produce different hashes. Stored in plain sight.	4
SECRET_KEY	The string every signature depends on. Leaking it means anyone can forge any token.	6, 12
Serializer	The translator between database rows and JSON, and where incoming validation lives.	3
Session	The other way to be remembered: a row on the server plus a cookie holding its id. Django's `/admin/` uses it.	1
sessionStorage	Like localStorage, but cleared when the tab closes. The usual place to park a message that has to survive a redirect.	9
Signature	The third part of a JWT. Guarantees the contents were not altered; guarantees nothing about secrecy.	6
Stateless	Keeping no memory between requests. Why proof must be carried on every one.	1

TERM	MEANING	CH.
Throttling	Limiting how often one caller may hit an endpoint. Absent here.	12
Transaction (atomic)	A group of database writes that all happen or none do. Wraps account creation.	3
Validator	One of Django's four password rules: similarity, length, commonness, all-numeric.	4
write_only	A serializer field that may come in but is never sent out. On `password` and `phone`.	3
XSS	Cross-site scripting: attacker text executed as code in the page. The risk that makes localStorage debatable.	12
401	I do not know who you are.	8
403	I know who you are, and no.	8

APPENDIX B

The Whole Flow on One Page

Print this one. Everything in the book, compressed to what you would want on the desk beside you.

Making an account

```
Admin fills /accounts
      |
      v  POST /api/register/  +  Authorization: Bearer <admin token>
IsAdmin  ------------------------------------> not an admin?  403
      |
      v
RegisterSerializer
  phone taken?     -> 400   username taken?  -> 400
  password weak?   -> 400   role not one of three? -> 400
      |
      v  @transaction.atomic
User.objects.create_user(...)   <- hashes the password
Profile.objects.create(user=..., phone=..., role=...)
      |
      v
201 Created  {id, username, email, first_name, last_name, role}
             (no password, no phone - both write_only)
```

Signing in

```
POST /api/login/  {phone, password}      (AllowAny — no token needed)
      |
      v
LoginSerializer: both fields present?          no  -> 400
      |
      v
Profile.objects.get(phone=...)                 none -> 400 "Invalid phone or password"
      |
      v
user.check_password(...)                       no  -> 400 "Invalid phone or password"
      |
      v
RefreshToken.for_user(user)     (nothing written to the database)
      |
      v
200  {message, user_id, username, role, tokens: {refresh, access}}
      |
      v
localStorage: lms_access_token = tokens.access
              lms_user         = {id, username, role}
```

Every request after that

```
fetch(...)  with  Authorization: Bearer <access token>
      |
      v
JWTAuthentication:  signature valid?  exp in the future?
      |                     no -> 401
      v
request.user = User.objects.get(pk=payload["user_id"])
      |
      v
permission class:  GET/HEAD/OPTIONS?  -> allowed for any signed-in account
                   otherwise role_of(request.user) in roles_that_may_write?
      |                     no -> 403 with the class's message
      v
the view runs
```

The numbers

SETTING	VALUE	WHERE
Access token lifetime	8 hours	`SIMPLE_JWT`
Refresh token lifetime	7 days (unused)	`SIMPLE_JWT`
Password reset link lifetime	1 hour	`PASSWORD_RESET_TIMEOUT`
Minimum password length	8 characters	`AUTH_PASSWORD_VALIDATORS`
Hash iterations	1,000,000 (PBKDF2-SHA256)	Django 5.2 default
Backend port	8001	`api.js` default, and the README's `runserver 8001`
Frontend port	5180	`vite.config.js`

The six endpoints

METHOD AND PATH	WHO	BODY IN	ANSWER
`POST /api/login/`	Anyone	`phone`, `password`	user, role, tokens
`POST /api/register/`	Admin	`username`, `email`, `password`, `phone`, `role`, names	201 with the new account
`GET /api/profile/`	Signed in	—	Your own details
`PATCH /api/profile/`	Signed in	Any of `first_name`, `last_name`, `email`, `phone`	Your updated details
`POST /api/change-password/`	Signed in	`current_password`, `new_password`, `confirm_password`	"Please log in again."
`POST /api/password-reset/`	Anyone	`email`	The same sentence either way
`POST /api/password-reset-confirm/`	Anyone	`uid`, `token`, `new_password`, `confirm_password`	"Please login again."

Who may write what

RESOURCE	ADMIN	TEACHER	STUDENT
teacher, student	write	read	read
course, lesson, assignment, enrollment, results	write	write	read
submission	write	write	read + create
accounts (register)	write	—	—

The eight sentences worth remembering

1. HTTP forgets you between requests, so proof travels with every one.
2. You log in with the phone number, because that is what the Profile is found by.
3. Passwords are hashed, never stored; the salt makes identical passwords look different.
4. A JWT is signed, not encrypted — anyone holding it can read it.
5. The role is not in the token, so a change in the admin applies to the next request.
6. 401 means "I do not know you". 403 means "I know you, and no".
7. The frontend's permission rules hide buttons; the server's rules are what stop anything.
8. A signed note cannot be taken back, which is why logging out only deletes your copy.